Journal of Network and Computer Applications 245 (2026) 104375

Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca

Review article

Edge-AI: A systematic review on architectures, applications, and challenges

Himanshu Gauttam [a], Garima Nain [b], K.K. Pattanaik [b,*], Paulo Mendes [a]

[a] Airbus, Central Research and Technology- 1XRC, Taufkirchen (Bavaria), Germany
[b] Wireless Sensor Networks Laboratory, Atal Bihari Vajpayee - Indian Institute of Information Technology and Management, Gwalior, India

ARTICLE INFO

Keywords:
AI
Deep learning
Cloud computing
Edge computing
Edge-AI
Edge-intelligence

ABSTRACT

The evolution of computing technologies and the generation of massive amounts of data fueled the development of *Artificial Intelligence* (AI), specifically *Deep Learning* (DL), solutions to extract key patterns from data, and the generation of insights and knowledge useful to achieve optimized service execution. Traditional cloud-based execution of DL solutions faces several challenges, such as latency, data privacy, and reliability, while trying to meet service requirements. In contrast, the limited computing and storage resources on the edge pose daunting challenges in executing resource-intensive DL solutions closer to the customer. This scenario led to the birth of an interdisciplinary research field named Edge-AI or Edge-Intelligence, aiming to mitigate the limitations of cloud and edge-based DL executions. In this context, this work proposes a reference layered Edge-AI framework to ensure the successful deployment of the Edge-Intelligence paradigm, encompassing three novel layers for the optimization of edge infrastructure, edge inference, and edge training. The work presents a detailed investigation and analysis of the schemes centered around the above-listed layers of the proposed Edge-AI framework. Furthermore, this work discusses potential application domains for Edge-AI, delving into a set of potential limitations, and ending up identifying future research directions in terms of Edge-AI infrastructure deployment, inference and training, which are functionalities needed to deploy and use robust, sustainable, and efficient intelligent edge networks.

Contents

1. Introduction 2
2. Related work 4
3. DEMMA: A reference layered Edge-AI framework 5
4. Methodology 6
5. Edge infrastructure optimization layer 6
6. Model inference optimization layer 9
   6.1. Binary offloading 9
      6.1.1. DNN inference accelerator in binary offloading 11
   6.2. Partial offloading 12
7. Model training optimization layer 14
   7.1. Solo (on device/edge) DNN learning in Edge-AI 16
   7.2. Distributed DNN learning in Edge-AI 17
      7.2.1. Federated learning 17
      7.2.2. Split learning 18
      7.2.3. Split federated learning 19
   7.3. Data availability based DNN training in Edge-AI 20
      7.3.1. Transfer learning in Edge-AI 20
      7.3.2. Continual learning in Edge-AI 20
8. Potential applications domains 23
9. Limitations and future directions 24
10. Conclusion 26
CRediT authorship contribution statement 27

* Corresponding author.
E-mail address: kkpatnaik@iiitm.ac.in (K.K. Pattanaik).

https://doi.org/10.1016/j.jnca.2025.104375
Received 6 April 2025; Received in revised form 13 October 2025; Accepted 27 October 2025
Available online 8 November 2025
1084-8045/© 2025 Published by Elsevier Ltd.

Declaration of competing interest 27
Acknowledgments 27
Data availability 27
References 27

1. Introduction

In recent years, the proliferation of the *Cyber Physical Systems (CPS)*, *Internet of Things* (IoT), and *Industrial IoT (IIoT)* paradigms led to the deployment of large sets of networks, each of which encompassing a large set of interconnected small devices (Vailshery, 2024; What, 2025; Cyber-Physical, 2019; Cyber, 0000). Each of these networks of connected devices produces a large amount of data, as is the case of interconnected cars generating 25 Gigabytes (GB) data per hour, or smart factory producing five Petabytes (PB) of data per week (Richter, 2017; Craig, 2018). All the data produced by these networks has to be processed in real-time/near-real-time to extract key patterns, which are needed to create meaningful insights and knowledge (Taylor, 2024). The development of efficient data processing systems, able to handle large amounts of data while ensuring the required low delays, leads to two significant technical challenges: (i) *How to process the data?* and (ii) *Where to process the data?*

A potential answer to the first question may be provided by *Artificial Intelligence* (AI), specifically *Deep Learning* (DL), solutions which have emerged as prominent candidates for processing massive amount of data aiming to extract meaningful information (Dong et al., 2021; Alzubaidi et al., 2021; Holdsworth and Scapicchio, 2024). However, the execution of DL tasks, such as inference and training, consumes a significant amount of computation, energy, and storage resources (Mathisen and D., 2020; Vipra and West, 2023), leading to the second question, i.e. where to run DL processes, taking into account their resource demand, and the need to reduce the latency of services.

Traditionally, DL solutions are executed in cloud centers, which are resource rich environments, aiming to cater to their resource intensive requirements (Azure, 2025; AI, 2025; Build, 2025). The cloud-based execution of DL tasks (also termed as Cloud-AI) requires the transmission of enormous data volumes from data sources to cloud centers, which imposes a significant burden on edge and core networks, resulting in: (i) increased delay, (ii) data privacy issues, (iii) higher bandwidth usage, and (iv) higher energy consumption. On the one hand, high delays and privacy violation may lead to service degradation and potential hazards to the end-user, since the transmission of large amounts of data to the cloud increases the risk of revealing sensitive users, products, and organization related information (Salih and Mohammad, 2024; Al-Dulaimy et al., 2024). On the other hand, high bandwidth and energy usages may lead to an increase of operational costs (Mitch et al., 2018), while the unavailability or intermittent availability of Internet connections in remote locations hampers the reliable performance of cloud-AI. Hence, it is clear that cloud-based execution of DL tasks is not a good fit for applications relying on factors such as real-time responsiveness, data privacy assurance, reduced energy consumption, and high reliability. This is the case of services such as smart health, intelligent transportation, smart manufacturing, and virtual and augmented reality (Kavitha et al., 2024; Nain and Samal, 2025).

The challenges faced by Cloud-AI may be mitigated by the usage of the *Edge Computing* paradigm, which allows the execution of computational tasks within the physical proximity of data producers, reducing communication latency and energy consumption, and avoiding the exposure of sensitive data (Khan et al., 2019; Zhang et al., 2021b; Qiu et al., 2020). In this context, an edge refers to a location other than a remote cloud data center that possesses some computational and storage resources (Nain et al., 2022). Within the edge, *Edge Devices* (EDs) are categorized, according to their computing and storage capacities, as: (i) micro/far, such as sensors or actuators; (ii) thin/mid, including single board computers; (iii) thick/near, such as on-premise servers. Thin/mid EDs and thick/near EDs also called *Edge Computational Nodes* (ECNs) and *Edge Servers* (ESs), respectively (Qiu et al., 2020; Matthew, 2020). The thick edge consists of powerful computing and large storage space (such as data centers, cell tower data centers etc.) responsible for bulk processing (such as factory level data storage and analysis). The thin edge provides the moderate level computation and storage facilities (such as single-board computer (e.g. Raspberry Pi), industrial PC equipped with Intel i-series processor etc.). Micro edge consist of sensors which are responsible for data generation and/or actuation (Matthew, 2020; Gauttam et al., 2022c). On the one hand, Edge Computing reduces data transmission delay, but simultaneously, the limited resources available at the edge, when compared to the cloud, increases the processing time of compute-intensive DL tasks, resulting in a higher task completion time. Furthermore, most edge devices (micro and thin EDs) are usually battery-powered embedded hardware with limited computational power, memory, storage capabilities, and strict energy budgets. It demands the development of energy aware solutions for the execution of computational tasks (Zhang et al., 2021b; Qiu et al., 2020; Gauttam et al., 2022c). Thus, with the resource-constrained nature of edge and the resource hungry characteristics of DL models, it becomes daunting to implement these models at the edge. Moreover, the deployment aspects of underlying edge infrastructure (such as locations of edge nodes, coordination among edge nodes, etc.) also impact the performance of DL task execution at edge (c.f. Section 5).

The mentioned challenges ushered to the investigation of a novel solution for the efficient execution of AI solutions at the edge, leading to the birth of a new interdisciplinary research paradigm, known as Edge-AI or Edge Intelligence. (Hua et al., 2023; Chang et al., 2021b; Murshed et al., 2021; Wang et al., 2020b).

The vision of Edge-AI is to transform AI into an ubiquitous system available to every person and organization everywhere, which can be achieved by pushing AI closer to the data producer and/or consumer (Sourabh, 2021). Edge-AI solutions aim to enable the efficient execution of AI tasks at resource-scarce edges while maximizing performance of DL tasks, such as inference accuracy, and minimizing costs, including computation, communication, storage, and energy costs. Edge-AI solutions are broadly grouped into two strategic groups: (i) solo (end/edge-only) execution, in which DL tasks are executed only locally at end device or edge device; (ii) collaborative execution (end-ED/ED-ED), in which DL tasks are executed based on the collaboration between a group of edge devices. Edge-AI solutions are starting to have an impact on enterprise ecosystems from big technology players such as Dell EMC, AWS, Google, Nvidia, IBM, which is an indication of its technical relevance (Sinha, 2024). The Edge-AI market is projected to be 356.84 billion by 2035 (Sharma and Kashyap, 2024).

However, despite many benefits, Edge-AI faces numerous key challenges. To start with, the solo strategy in which DL tasks are executed only on end devices or edge devices may result in increased task execution time and device energy depletion due to limited availability of resources. In addition, executing the large-size DL model with limited memory and storage on the end device is challenging. DL model resizing solutions, such as model compression, attain the near-real-time execution of the model within the end device at the cost of reduced model performance (e.g. lower accuracy). These limitations may be mitigated by adopting a collaborative strategy in which DL tasks are executed among a group of edge devices. With this strategy, DL tasks are distributed among a set of collaborative edge devices, in order to efficiently utilize available resources while catering to the application

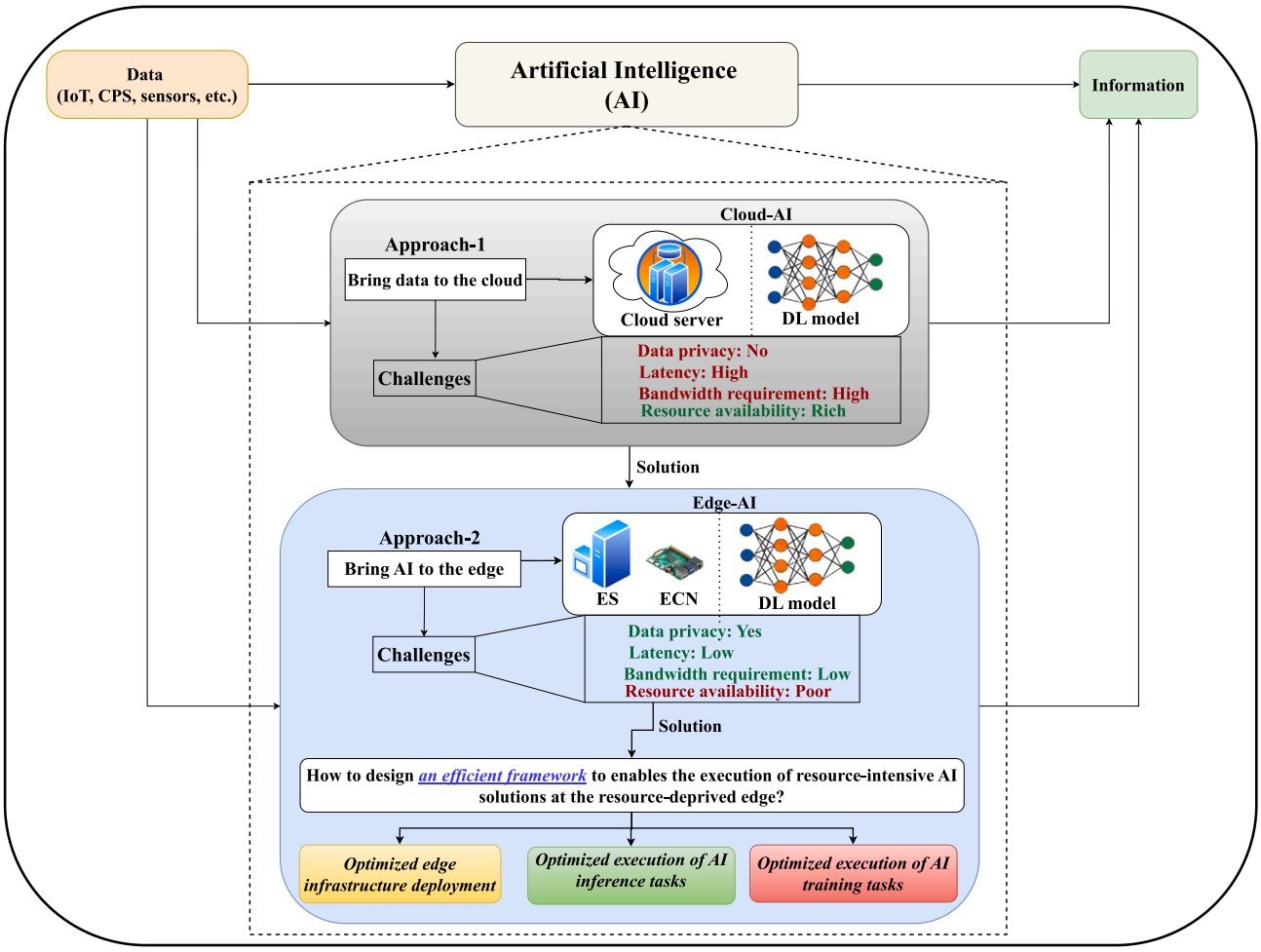


Fig. 1. Evolution of Edge-AI paradigm.

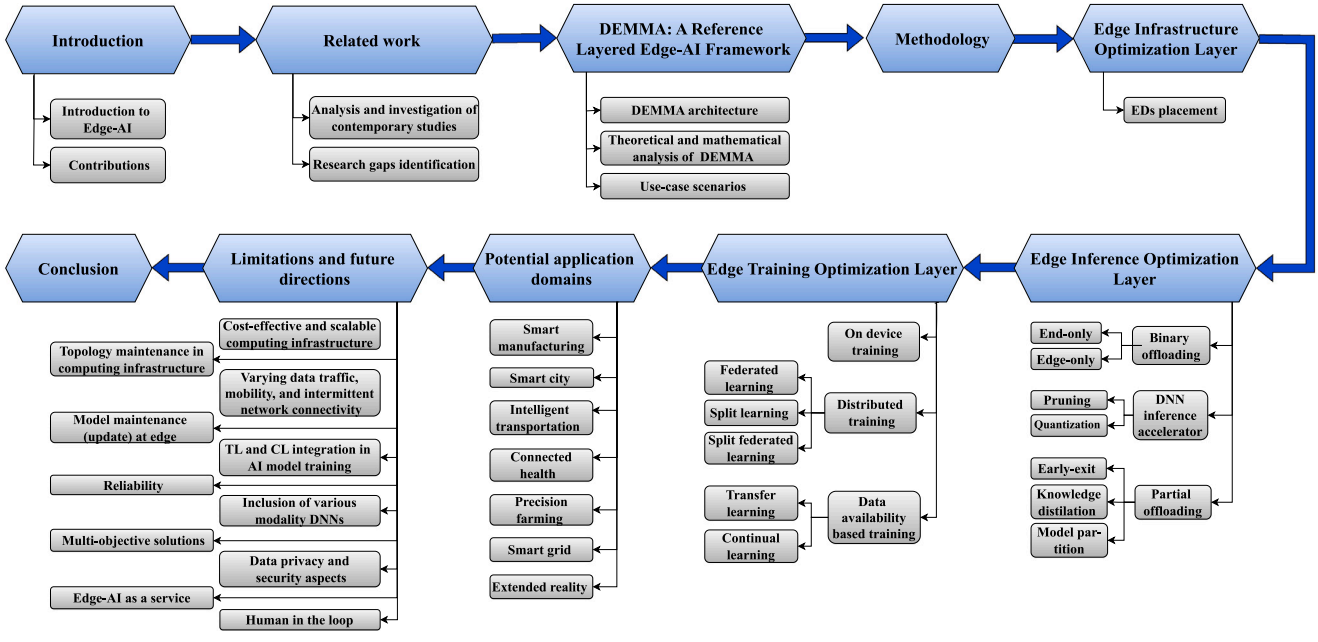


Fig. 2. Organization and layout of the study.

requirements, such as low latency and costs, and data privacy. The key challenges of an Edge-AI collaborative strategy encompasses the selection of suitable and trustful edge devices, and the efficient distribution of tasks among them, while preserving data privacy. However, the

Table 1
Comparison from the existing surveys.

Reference	Edge-AI architecture	Edge infrastructure	Edge inference			Edge training					
			DNN tasks characteristic	Offloading		Offloading				Data availability	
				Binary	Partial	Binary	Partial			TL	CL
							FL	SL	FSL		
Wang et al. (2020b)	×	×	×	×	✓	×	✓	×	×	×	×
Shi et al. (2020)	×	×	×	✓	✓	×	✓	×	×	×	×
Deng et al. (2020)	×	×	×	✓	✓	✓	✓	✓	×	×	×
Chang et al. (2021b)	×	×	×	✓	✓	×	✓	✓	×	✓	×
Xu et al. (2021)	×	×	×	✓	✓	✓	✓	✓	×	✓	×
Shuvo et al. (2022)	×	×	×	✓	✓	×	×	×	×	×	×
Yao et al. (2022)	×	×	×	✓	×	×	✓	×	×	✓	×
Baccour et al. (2022)	×	×	×	✓	✓	×	✓	×	×	×	×
Nain et al. (2022)	×	×	×	✓	✓	×	✓	✓	×	✓	×
Rosendo et al. (2022)	×	×	×	✓	✓	×	✓	✓	×	✓	×
Duan et al. (2022)	×	×	×	✓	✓	×	✓	✓	×	×	×
Tuli et al. (2023)	×	×	×	✓	✓	×	×	×	×	×	×
Hua et al. (2023)	×	×	×	✓	✓	×	✓	×	×	×	×
Ahmad et al. (2023)	×	×	×	✓	✓	×	✓	×	×	×	×
Jouini et al. (2024)	×	×	×	✓	✓	×	×	×	×	×	×
Wang et al. (2024b)	×	×	×	✓	✓	×	✓	✓	×	✓	✓
Gill et al. (2025)	×	×	×	✓	×	×	×	×	×	×	×
Proposed	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

presence of heterogeneous devices, intermittent network connection, and varying resources, among other conditions, imposes additional complexity to design efficient collaborative Edge-AI solutions.

Based on all these challenges, the efficient deployment of the Edge-AI paradigm in different scenarios benefits from a systematic survey and analysis of different types of potential solutions. However, the existing contemporary studies lack a concrete discussion of a unified framework for Edge-AI ecosystem. This work encompasses such detailed survey and analysis, based on which we propose a reference layered Edge-AI framework to ensure the efficient execution of Edge-AI solutions. The proposed reference architecture enables the investigation of various Edge-AI challenges on a layer/level granularity, and allows the investigation of dependencies among various layers involved in the execution of Edge-AI solutions. For instance, an optimally deployed edge infrastructure results in reduced communication costs for the execution of DL tasks using collaborative solutions. Fig. 1 depicts the evolution of Edge-AI paradigm.

To the best of our knowledge, this is the first work that provides a detailed systematic layer-wise analysis to accomplish an efficient Edge-AI framework design. It includes the investigation of key challenges, analysis of existing solutions and their limitations, and recommendations towards potential future direction in designing an optimized, robust, and sustainable Edge-AI framework. The key contributions of the proposed work are as follows:

- A reference layered Edge-AI framework encompassing key components for infrastructure deployment, inference and training optimization, technical challenges and techniques to address them.
- A theoretical proof followed by the mathematical proof validates the rationale behind the selection of key components of Edge-AI framework.
- A comprehensive taxonomy related with edge infrastructure deployment, encompassing issues such as deployment of edge devices and load balancing among them.
- Analysis of different inference and training execution strategies, utilized to attain near-real-time responsiveness, reliability, reduced energy consumption, etc., at the edge.
- Discussion of potential Edge-AI application domains, including open research challenges and possible future research directions.

The structural organization and layout of the study are presented in Fig. 2 for a clear overview, outlining the key components, sequential structure, and flow of the research work. The work is organized as follows: Section 1 gives an introduction to the study. Section 2 provides a comparative analysis of similar surveys and review papers, which illustrate the need to develop a reference model for an Edge-AI framework. In Section 3, we propose a reference layered Edge-AI framework, called DEMMA, and provide a mathematical justification for its adoption. The methodology of the survey is illustrated in Section 4. Section 5 analyses the optimized edge infrastructure deployment technique proposed on DEMMA. The schemes utilized in DEMMA to enable the execution of DL inference and training tasks at the edge are discussed in Sections 6 and 7. Section 8 illustrates the potential Edge-AI application domains. The limitations of existing Edge-AI solutions and future research directions are elaborated in Section 9, followed by Section 10, concluding the paper.

2. Related work

This section presents an investigation of seventeen contemporary Edge-AI surveys in the view of Edge-AI framework challenges (c.f. Fig. 1), illustrated in Table 1. The contemporary literature lack a concrete discussion on a unified framework for Edge-AI ecosystem. It serves as the motivation for proposed DEMMA architecture, which provides a holistic view of Edge-AI system. It provides a broad abstracted view as well as specific detail at each component level, i.e., details of a system from the high-level component level perspective towards a low-level implementation details. The existing state-of-the-art coexisting surveys discussed the various Edge-AI issues, including communication and computational challenges in executing AI task at edge, edge inference and training techniques, distributed and collaborative Edge-AI schemes, distributed DL in edge-cloud continuum, etc. In summary, the discussion of existing studies is centered around Edge-AI inference and training solutions. Edge inference optimization schemes aim to reduce the execution time of DL inference tasks at the edge while catering to application-specific performance measures. The existing survey explored various solutions, including offloading schemes (binary and partial offloading), DNN inference accelerators (such as pruning, quantization, etc.), and hybrid solutions to attain optimized execution of DL inference tasks. However, Wang et al. (2020b), Yao et al. (2022) and Gill et al. (2025) did not explored the Edge-AI inference optimization schemes from binary and partial offloading perspective. Furthermore, the offloading decisions and resulted performance metrics

are highly affected by the task characteristics such as type of DL tasks, number of DL tasks, arrival pattern of DL tasks, etc., which is not included in the existing surveys. Edge training optimization solutions aim to accomplish the efficient execution of compute-intensive DL-training tasks at the resource-deprived edge. The recent surveys, except (Shuvo et al., 2022; Tuli et al., 2023; Jouini et al., 2024; Gill et al., 2025), leverage the binary and partial offloading solutions to achieve optimized DL training at the edge. Binary offloading-based schemes (Deng et al., 2020; Xu et al., 2021) leverage the various DNN accelerator solutions, enabling the execution of DL tasks at the edge. Moreover, the partial offloading empowered DL training approaches empower the distributed and collaborative DL training at the edge, comprising the discussion of schemes such as Federated Learning (FL) and Split Learning (SL). However, the analyzed surveys did not investigate the recent research solutions, encompassing the characteristics of SL and FL to enhance the efficiency of distributed DNN training, termed as Split Federated Learning (SFL). In addition, updating a DL model to address the dynamism and adaptability aspects with limited available data (due to limited storage at edge and/or data governance regulation) is challenging at edge. Transfer Learning (TL) and Life Long Learning (LLL) such as Continual Learning (CL) are utilized to achieve the same, which are not discussed in the coexisting surveys in detailed fashion. In existing studies, thirteen out of seventeen surveys discussed a wide range of Edge-AI inference and training schemes.

However, all of the existing studies the ignore the importance of optimized edge infrastructure deployment which may have a significant impact on the DL task completion times, must be included in a reference Edge-AI framework to attain the efficient execution of Edge-AI solutions. Similarly, the recent surveys did not examine the impact of DNN task characteristics on Edge-AI inference solutions. Moreover, the existing surveys did not explore the dynamism and adaptability issues in Edge-AI training solutions. Besides, None of the existing surveys have examined Edge-AI from a system-level perspective to answer the question such as: (i) what are the fundamental components of an Edge-AI ecosystem? (ii) what is the relationship among the components of Edge-AI system? (iii) what are the key challenges and remedies of the Edge-AI framework at the component level? The above mentioned limitations form the foundational basis for the work presented in this article, encompassing fundamental Edge-AI components, their interrelationships, and a detailed discussion of associated challenges along with potential solutions, to accomplish the design of an efficient Edge-AI framework.

3. DEMMA: A reference layered Edge-AI framework

This section describes our proposal for a reference layered Edge-AI framework able to ensure the successful deployment of Edge-AI solutions, including a mathematical validation of its adoption. The proposed layered Edge-AI framework, which we call DEMMA (c.f. Fig. 3), comprises the following four layers:

- [**D**] **Data acquisition layer** (Layer 1) accomplishes the collection of multi-sensor heterogeneous data from the surrounding environment and end devices. This layer interacts with layer 2 to transfer the collected data to edge devices in which DL inference and training will be conducted.
- [**E**] **Edge infrastructure optimization layer** (Layer 2) takes care of edge devices (ECN/ES) placement and the association of end devices (such as IoT nodes) with the most suitable edge devices, based on some performance metrics such as delay reduction. In other words, this layer aims to setup an infrastructure of edge devices, ECN/ES, aiming to reduce the cost of accessing the DL models needed by specific services while maximizing the utilization of resources at the edge. An optimized edge infrastructure reduces the data transmission delay from end devices to edge devices and response (insights) transmission delay from edge devices to end devices. The edge infrastructure managed by this layer supports the optimization of inference and training (layer 3), essential for the execution of applications at layer 4.
- [**MM**] **Model inference and Model training optimization layers** (Layer 3) comprise the mechanisms that enable the efficient execution of DL-tasks (such as DL model inference and training tasks) hosted at EDs. These solutions reduce the data processing delay (such as minimizing the execution time of DL model inference and training tasks) while preserving the required model performance (such as accuracy). Thus, the above discussion concludes that optimized edge infrastructure deployment and optimized execution of DL tasks at the deployed edge enrich the application performance. Layer 3 interacts with layer 2 to access the resources (computational and storage) for the execution of DL tasks. The interaction of layer 3 with layer 4 provides the response(s) for the requested DL tasks to the respective applications.
- [**A**] **Application layer** (Layer 4) represents the potential applications that can be enriched using Edge-AI, such as smart city, connected health, smart industry, precision farming, including many other applications. It interacts with layers 3 to enable the efficient execution of DL tasks.

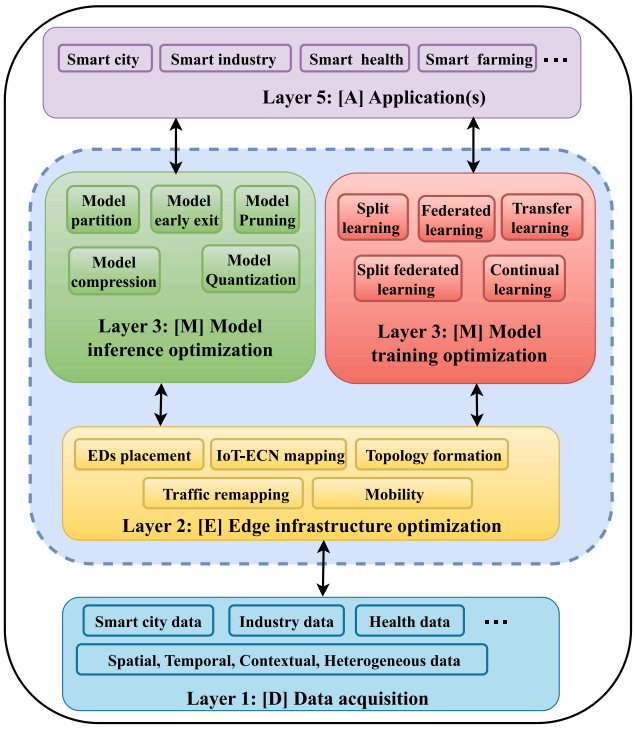


Fig. 3. DEMMA: A layered Edge-AI framework.

The proposed reference architecture enables the investigation of various Edge-AI challenges on a layer/level granularity. In summary, service providers use the application layer (layer 4) to make their services (e.g., smart city, smart industry) available. Each of these services makes use of the inference optimization and training optimization (layer 3) models for the implementation of their DL tasks in resource-constrained EDs, which are hosted and executed at the edge infrastructure (layer 2) to process the data captured by data acquisition functionality (layer 1).

Use-case scenarios:

For instance, an object detection task (inference task) of an autonomous driving application (layer 4), end node (vehicle) initiates the data (images) capturing (at layer 1), which is transmitted to the

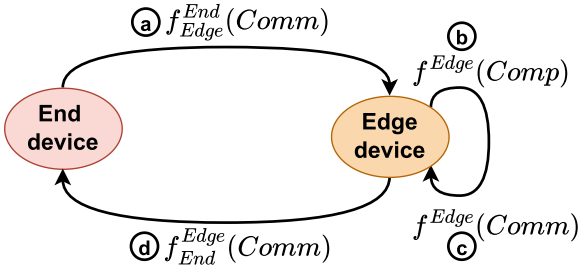


Fig. 4. Execution cost of a DL-task.

respective ED. It comprises the significant data transfer delay from the vehicle to the ED. Layer 2 functionalities, such as efficient ED placement and balanced mapping, reduce the data transfer delay. Moreover, efficient placement of EDs and balanced mapping also reduced the data processing delay at EDs. Next, ED(s) utilize layer 3 functionality, such as split learning process received data to cater the application QoS, such as near-real-time responsiveness. Similarly, a smart city application (layer 4) may employ the functionality offered by layer 3, such as federated learning, to train a generalized DL model through the collaboration of optimally deployed edge nodes (layer 2) without sharing the raw data (layer 1). Moreover, in a smart factory floor, various sensor node captures the event-related data (layer 1) and transmit it to the respective edge node (layer 2). The received data is processed (layer 3) by the edge node locally/collaboratively based on the complexity of the processing task to extract the meaningful information, which is transmitted back to the sensor nodes for optimized decision making. The analogy applies to the other application use cases such as smart farming, smart manufacturing, intelligent health care systems, etc. (Nain and Samal, 2025; Vaigandla, 2025; Mahmoud et al., 2024; Gauttam et al., 2023).

The survey and analysis described in this paper is related to the edge infrastructure optimization layer (layer 2), the model inference and training optimization layer (layer 3), as depicted in a dashed blue box in Fig. 3.

Before diving into the detailed analysis of how to optimize edge infrastructure to increase the performance of inference and training models, it is important to understand how successful could be the adoption of this reference layered framework. With this in mind, we provide a mathematical justification behind the intuition of adopting a layered architecture to represent Edge-AI framework. The DL task (inference/training) execution cost involves the data transmission from end device to edge device, task execution cost at the edge device, and the cost of transmitting the results from edge device to end device, respectively, as depicted in Fig. 4, and formulated in Eq. (1), which computes the Completion Time (CT) of a DL task (inference task) T,

$$f^{CT}_{T} = f^{End}_{Edge}(Comm) + \gamma \times (f^{Edge}(Comm)) + f^{Edge}(Comp) + f^{Edge}_{End}(Comm) \tag{1}$$

where, $f^{End}_{Edge}(Comm)$, $f^{Edge}(Comm)$, $f^{Edge}(Comp)$, and $f^{Edge}_{End}(Comm)$ represent the data transmission cost from end device to respective edge device, data transmission cost among edge devices, computation cost to process the data (such as execution cost of a DL inference task) at edge device(s), and data transmission cost from edge device to end device, respectively. In Eq. (1), the variable γ has a binary value that depends on the data processing techniques at the edge. For instance, if data processing task is accomplished by a single edge device then value of γ is set to zero and in others cases (i.e. collaborative processing) the value of γ is set to one.

Data communication cost from end device to edge device and vice-versa (represented as **a** and **d** in Fig. 4) is affected by the deployment of edge infrastructure (layer 2), which includes operations like the placement of EDs and the mapping of end nodes to respective EDs. Edge infrastructure deployment also affects the communication cost among the collaborative devices (represented as **c** in Fig. 4). The computing and communication cost (represented as **b** and **c** in Fig. 4) at edge is affected by schemes, such as local execution, and collaborative execution, which are utilized to execute the DL task (layer 3).

It is evident from the above discussion that DL-task execution cost is given as the summation of communication and computation costs (refereed as **a**, **b**, **c**, and **d** in Fig. 4). Adopting a layered architecture framework associates these costs with respective layers, facilitating layer-wise investigation and analysis of various issues (such as ED placement and mapping at layer 2) to attain optimal performance. In other words, deploying optimized edge infrastructure and designing efficient schemes to execute the DL tasks at the edge reduces the overall execution cost of DL-task, leading to efficient Edge-AI framework design. Thus, the above discussion justifies the adoption of a layered architecture for Edge-AI framework representation.

4. Methodology

This section elaborates on the methodology followed in the work, providing an in-depth understanding of the article collection. There are numerous methodologies for the article collection and selection, such as systematic literature review (Paluri et al., 2025), Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) (Zhang et al., 2017), etc. The study adopted the PRISMA methodology to ensure a transparent, systematic, and reproducible process for identifying, screening, and selecting relevant research articles. A comprehensive search for article collection is conducted across IEEE Xplore, Scopus, etc. In this context, the main keywords identified for search are “edge AI”, “edge intelligence”, along with “edge infrastructure”, “edge server placement”, “federated learning”, “splitting”, “partitioning”, “model update”, “distributed inference” or “distributed training” within the article title, abstract, and keywords. The retrieved records are fed to a four-phase PRISMA process of identification, screening, eligibility, and inclusion as shown in Fig. 5.

In the identification phase, a total of 945 articles were collected from the article databases, and 41 were removed due to a lack of relevance. The remaining 904 records or articles are first screened for duplication. Hence, 125 were removed as duplicate articles and further screened based on Year and Language. A total of 513 articles were sought for eligibility, considering English language and 2017–2024 publications. During the eligibility phase, 294 articles were removed due to lacking publication in peer-reviewed journals (such as conference, reviews, and chapters). The remaining 219 were reviewed for abstract and full text, and 43 were removed due to irrelevance, out of scope, lacking methodological details, or inaccessible full-text. Finally, in the inclusion phase, only those studies that met all criteria and contributed significantly to the research objectives were retained for detailed analysis. Therefore, the study included 176 articles. In addition, some suitable conference papers were added due to their high relevance. This PRISMA-based approach ensured methodological rigor, minimized selection bias, and enhanced the reliability of the literature synthesis.

5. Edge infrastructure optimization layer

This section discusses the challenges associated with the second layer of DEMMA (c.f. Fig. 3). It investigates issues such as the impact of EDs (such as ECN/ES) placement, mapping of end nodes among EDs, topology formation etc., to design an optimized edge infrastructure. In a network of connected end devices (such as IoT nodes), each node acts as a candidate location for ED (varying from ECN to ES) placement. Similarly, each ED serves as a potential candidate to map end nodes. Thus, the number of possible placement and mapping strategies are combinatorial. Thus, efficient edge infrastructure deployment by

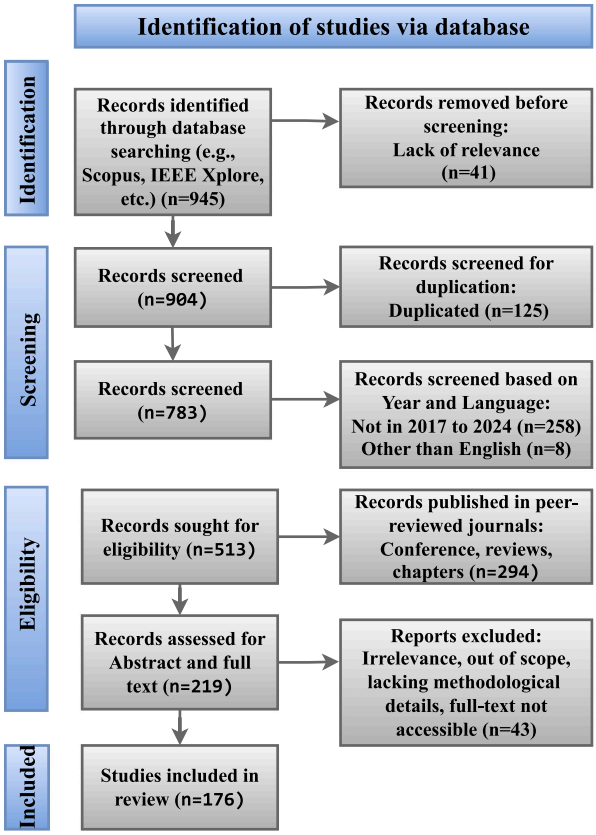


Fig. 5. PRISMA-based inclusion and exclusion towards article selection.

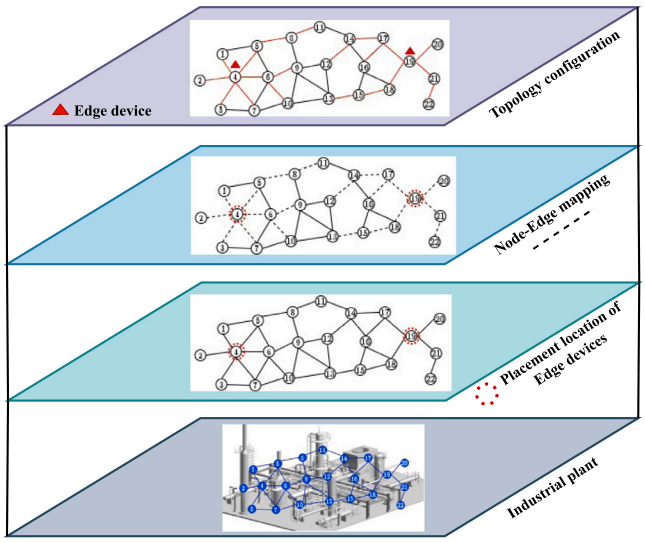


Fig. 6. Edge infrastructure deployment procedure (Gauttam et al., 2022c).

selecting an optimal placement and mapping strategy among a large set of choices is challenging since it needs to minimize the delay and energy consumption and maximize the efficient utilization of edge resources. It initiates the need for an efficient solution to cope with an edge infrastructure deployment scheme in a reasonable time. Fig. 6 summarizes the key steps involved in edge infrastructure deployment (Gauttam et al., 2022c). The network graph of end nodes is fed as input to the edge infrastructure deployment scheme. The first step of edge infrastructure deployment comprises the identification of optimal/near-optimal locations for EDs placement. The placement of EDs directly affects the data transmission delay (refer to Eq. (1)) and energy consumption of end nodes. It also affects the communication cost among EDs. The next issue involves the balanced mapping of end nodes among the EDs, which impacts the data transmission delay, data processing delay (for DL task execution), resource utilization at EDs, and energy consumption. The last step utilizes the placement and mapping information to generate the final topology at edge. The edge layer acts as a resource provider for hosting and executing the DL tasks and optimized deployment of edge infrastructure affects the performance of DL task. For instance, efficiently deployed EDs can reduce the communication cost among collaborative devices, reducing the cost of DL task execution. Similarly, balanced node-ED mapping attains efficient resource utilization at EDs and lower the waiting delay of DL tasks at EDs. It minimizes the overall DL-task execution cost. Thus, optimized edge infrastructure significantly affects the performance of DL-task execution. It also serves as the motivation to integrate edge infrastructure optimization as one of the key layer in the proposed DEMMA framework.

Edge infrastructure plays a vital role in ensuring application QoS. For instance, an autonomous vehicle (end nodes) comprises the latency-critical tasks such as obstacle detection and avoidance, traffic sign recognition, etc. The autonomous vehicle captures the sensory data from the surroundings and transmits it to the respective ED. In such scenarios, the inefficient placement of EDs can increase the data transfer delay, hampering the real-time responsiveness of the application. In addition, it also increases the offloading overhead (transmission, processing, and waiting delay) among the EDs. Furthermore, the unbalanced mapping of end nodes (vehicles) among the EDs leads to under-utilization/over-utilization scenarios. It reduces efficient resource utilization, deteriorating the application QoS and customers' Quality of Experience (QoE). The performance of edge infrastructure deployment schemes are evaluated in terms of the delay, energy consumption, data traffic distribution among EDs, load balancing, resource utilization, etc

Meng et al. (2017) utilized a variant of k−facility problem to model the cloudlet placement to identify wireless Access Points (APs) to place cloudlets to minimize the delay between users and cloudlet. The scheme initially deploys the cloudlet at k random locations and assigns the APs to the cloudlets with minimum delay. Further, an iterative scheme identifies the new set of candidate APs for cloudlet placement and determines the new delay. The new APs are selected for cloudlet placement if the current delay is less than the previous delay. In this way, the solution iterates to obtain a set of k AP with a smaller delay. Heaviest-Access-Point-First (HAF) and Density-Based-Clustering (DBC) schemes are devised for efficient cloudlet placement in Wireless Metropolitan Networks (WMAN) to minimize access delay and attain optimized user-to-cloudlet assignment (Jia et al., 2017). The locations of the heavily loaded APs are selected for the cloudlets placement in HAF. On the other hand, DBC determines the APs with dense user areas for cloudlets' placement and employs a relative distance metric to achieve optimized user-to-cloudlet assignment. Furthermore, DBC eliminates the redundant nodes during the cloudlet placement location determination, minimizing the access delay and enhancing resource utilization compared to HAF. Li and Wang (2018) exploited the Particle Swarm Optimization (PSO) technique to design an energy-aware ES placement strategy in a Mobile Edge Computing (MEC) environment. The placement problem aims to determine the placement locations of ESs that can reduce overall energy consumption and meet the access delay requirement.

Mixed integer linear programming and benders decomposition are utilized to identify the locations of APs to place the cloudlets and associate the users with respective cloudlets to cater to the delay requirements of each task and minimize energy consumption (Yang et al., 2019). Zeng et al. (2018) investigate the issues of efficient and economical deployment of ESs in WMAN and devised Simulated Annealing

(SA) and Graph Dominating Set (GDS) based solutions for minimizing the number of ESs while meeting QoS requirements. Binary-based differential evolution cuckoo search (BDECS) algorithm is exploited to determine the cloudlet deployment location among a large number of APs in IoT network to minimize the latency and cloudlet deployment cost (Wang et al., 2020a). The authors also investigate the fault tolerance aspect for cloudlet placement in IoT networks. Fan and Ansari (2019) developed a Lagrangian heuristic scheme for cost-efficacious cloudlet placement to reduce cloudlet cost and average end-to-end delay in MEC. Li et al. (2018c) employed k-means clustering approach for ES placement in MEC for the reduction of average completion time.

Guo et al. (2020) discussed an approximate approach comprising k-means clustering and Mixed-Integer Quadratic Programming (MIQP) to place a fixed number of edge clouds at the candidate locations to minimize communication delays and balanced workload among edge clouds. A Mixed-Integer Programming (MIP) solution is presented to place ESs at strategic locations in MEC that minimizes access delay from end-user to ES and obtains a balanced workload among ESs (Wang et al., 2019). The ES placement problem is modeled as a multiobjective constraint optimization, identifying the strategic placement locations of ESs to balance the workloads of ESs and for reducing access delay between the industrial control center and ES (Kasi et al., 2020). The work presented a comparative performance analysis among Hill Climbing (HC), SA, and Genetic Algorithm (GA) in the context of optimal ESs placement. Lähderanta et al. (2021) developed an ES placement solution, PACK, which reduces the distance between ES and their associated APs and addresses the load balancing among ESs.

Chin et al. (2020) designed a k-clustering-based ES placement for network traffic reduction. The clustering procedure initializes k nodes having the highest request as ESs and assigns the request of remaining nodes among ESs using the shortest hop distance. Further, it selects a new node as ES in each cluster and assigns the requests of the remaining cluster nodes to it. The process is executed iteratively until the traffic load difference between the previous and current iteration is less than a predefined threshold. Discrete Differential Evolution (DDE) scheme accomplishes the optimized fog node placement in a smart factory setting to reduce response time and attain load balancing (Wang et al., 2020c). Wang et al. (2022b) described a Binary-based Gray Wolf Genetic Scheme (BGWGS) for optimized ES deployment to reduce deployment cost and attain load balance. The scheme also discussed a fault-tolerant server deployment mechanism that replace the faulty server in time, when an ES fails, to ensure the reliability of routine workflow in smart manufacturing. Jiang et al. (2020) devised an improved k-means clustering mechanism for ESs deployment in smart manufacturing environment to reduce the delay and deployment cost. A Mobile Edge Server (MES) deployment scheme is discussed to deploy the MES adjacent to the 5G Base Station (BS) to upgrade the performance of the specific application (Li et al., 2022a). The scheme utilizes the improved k-Means clustering to determine the number of ESs and their respective placement locations, which efficiently reduce the network delay, energy consumption and cost of the ESs. It assigns the computation tasks of end nodes' to the distance-first ES. Cao et al. (2020) devised an efficient scheme to explore the placement of heterogeneous ES under mobile user scenarios. The scheme includes a offline and online phase. In the offline phase, an integer linear programming (ILP) is used to determine the optimal locations to place the ES. The online phase leverages a game theory based approach to remap the data traffic of base stations to handle users mobility.

Chang et al. (2021a) described an ES deployment mechanism to support low-latency services to end users, while minimizing the deployment cost for service providers, in a metropolitan area. It formulates the server placement problem as a multi-objective mixed integer quadratic optimization problem, to achieve the trade-off among the deployment cost, number of hand-offs, and transmission delay. A divide-and-conquer based heuristic multi-objective optimization scheme is proposed to attain the above listed objectives. Chen et al. (2022a) addresses the concern of efficient deployment of ESs under a certain budget, while handling explosive growth of data scale and user base. It discussed a preference-aware ES placement approach to minimize query latency and offers balanced workload distribution among ESs. The ES placement is formulated as a p-center problem using multi-objective optimization. The authors proposed a Quadratic Integer Programming (QIP) for small-scale data sets and a heuristic algorithm termed as Tabu search with k-means and Genetic algorithm (TAKG) for large-scale data sets. Li et al. (2021) described the mechanism to appropriately position the ESs to ensure the access delay and maximize edge providers' profit. The profit model is formulated as a function of access delay and energy consumption and a PSO based scheme is designed to attain maximized profit. Ling et al. (2022) discussed an efficient ES placement mechanism to select optimal locations for the placement of ESs to minimize network latency and energy consumption. The authors adopted a Graph Convolution Network (GCN) based model to predict the network traffic. Further, edge server placement problem is modeled as a constrained optimization problem and utilized a PSO based solution to identify the optimal locations to place ESs to attain minimized delay and energy consumption. Gauttam et al. (2022c) designed a Cost-aware ECN Placement (coECNP) scheme for optimal topology formation in EC environment for minimizing the delay of IoT nodes. It utilizes a density-based approach to identify the location for ECNs placement and an iterative approach to obtain the balanced mapping of IoT nodes among ECNs. A traffic-remapping based solution extends the same to achieve low latency and balanced data traffic among ECNs in varying data traffic scenarios (Gauttam et al., 2022b). Two-stage solution for mobility-aware ES placement to maximize query throughput and minimize response time is discussed in Chen et al. (2023b). The devised solution comprises an offline and online phase to deploy ESs. The offline phase uses an NSGA-based heuristic search algorithm to identify the optimal placement strategy. On the other hand, a game theory-based online heuristic correction mechanism adapts to the changes in mobile users' points of interest. Jiang et al. (2023) described an intelligent solution to select the optimal ES placement locations for minimizing the service cost by adjusting the layout of ESs while considering the dynamic network environment. The Markov Decision Process (MDP) is used to model the dynamic ES placement problem as a sequential decision process, and a Deep Reinforcement Learning (DRL) mechanism is utilized to solve the same.

Mazloomi et al. (2022) designed the ES placement as a joint combinatorial optimization problem comprising the placement of ESs and allocation of base stations to the respective ESs. The solution proposed a novel Reinforcement Learning (RL) framework, which comprises a temporal difference learning algorithm to obtain a scalable ES placement solution. Asghari et al. (2023b) devised an efficient dynamic ES placement solution in MEC environment to minimize the delay and improve resource utilization. To begin with, the clustering scheme divides the region into multiple sub-regions, and the Red Deer algorithm is adopted to identify the optimal location to place the ES within each sub-region. Further, the authors utilized the Markov process and Q-learning algorithms to achieve global convergence and cater to the needs of mobile users. Further, the authors employ a combination of Butterfly Optimization (BOA) and Coral Reefs Optimization (CRO) to address the latency and energy aware ES placement (Asghari et al., 2023a). Mehta and Verma (2024) developed a QoS-aware ES placement mechanism for the use-case of collaborative predictive maintenance solutions in industrial IoT. The work adopted k-means++ based solution to identify initial ES placement locations and designed a Whale Optimization Algorithm (WHA) to obtains the optimal ES locations. Tiwari et al. (2024) formulated the ES placement problem as *0–1* knapsack and discussed a PSO based solution to improve load balancing, ES utilization and reduce energy consumption, respectively. Vali et al. (2024) devised a greedy approach to address the ES placement problem in MEC scenarios to obtain the load balancing among ESs and minimize the network traffic between them. The approach recursively divides

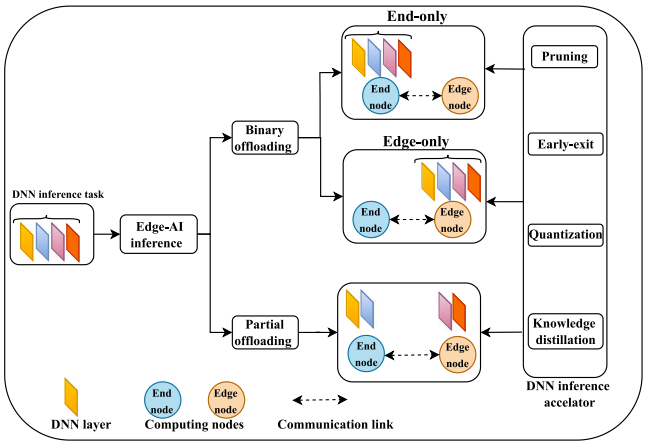


Fig. 7. Edge-AI Inference schemes.

the MEC graph until an even distribution of workload is achieved within each cluster to minimize the standard deviation of traffic among them. Asghari et al. (2024) discussed an Energy-aware server placement using the Trees Social Relations (TSR) algorithm to obtain the global placement locations for ESs in MEC networks to minimize the delay and energy consumption. The ES positioning and allocation problem in MEC is addressed using multi-objective solution, employing NSGA-II algorithm, to minimize the average delay and workload variance among ESs (Ghasemzadeh et al., 2024). Table 2 summarizes the various aspects of above discussed optimized edge infrastructure deployment solutions.

Discussion: Efficient edge infrastructure deployment is a key component to realize the full potential of Edge-AI solutions, which is directly affected by ES placement, IoT-ES mapping, data traffic distribution among ES, etc. The recent studies discussed various ES placement schemes with the objectives of delay minimization, load balancing, reduction of energy consumption, deployment cost minimization, and efficient resource utilization, respectively. It enable the usability of these solutions across various applications, including smart city, connected health, precision farming, etc., while considering application QoS. However, a limited studies discussed the solution which considers the trade-off among above listed performance metrics. Most of the work explores the ES placement solutions under the assumption of fixed number of ES, which raise the scalability concerns under the rapid growth of IoT networks and amount of data. It limits the applicability of existing schemes in scenarios such as smart cities, intelligent transportation systems, and similar applications. Moreover, the reliability and fault tolerance aspects needed to include in ES placement schemes, to cater the reliability requirements of applications such as connected health, smart manufacturing, critical infrastructure, etc. Table 2 represents the summary of edge infrastructure deployment schemes.

6. Model inference optimization layer

This section discusses the third layer of the DEMMA architecture, comprising the solutions for efficient execution of DNN inference task[1] at the resource-constrained network edge. Fig. 7 depicts the various approaches, which enable the efficient DNN inference execution in Edge-AI paradigm. DNN inference is defined as the procedure of executing the input data through a sequence of DNN layers to obtain the final result. In DNN inference task, each tile represents a DNN layer. In the autonomous driving use-case scenario realm, the model inference optimization layer handles the execution of the DNN inference task (such as object detection, lane detection, etc.). It involves processing the captured data with a pre-trained DNN model to extract key insights.

Edge-AI inference schemes employ binary and partial task-offloading concepts to execute the DNN inference task at the edge. Binary-offloading-based solution (Section 6.1) comprises: (i) End-only: DNN inference task is executed at end-device (such as IoT node) and (ii) Edge-only: offloads the DNN inference task at edge node (ranging from ECN to ES), respectively. Partial offloading solutions (Section 6.2) split the DNN inference task among collaborative devices (varying from end device to edge devices). Moreover, binary and partial offloading solutions also integrates the DNN inference accelerators for performance enhancement. Furthermore, the selection of the DNN task inference execution scheme is highly dependent on the application requirements (such as latency, reliability, etc.) and compute and storage resource availability, and network connectivity. For instance, an end-only scheme (under binary offloading) utilizes the onboard available resources to execute the inference task. It ensures data privacy and eradicates network dependency constraints. However, it also demands the availability of higher computing and storage resources onboard to ensure application QoS. Similarly, edge-only schemes cater to higher resource demands but increase data transfer latency from vehicle to ED and raise data privacy issues. The partial offloading solutions cooperatively leverage the available resources at the vehicle and EDs to assure resource demands, application QoS, and preserve data privacy through intermediate data transmission among the cooperative nodes. However, it imposes the additional overhead for offloading decisions. The performance of the Edge-AI inference solutions are determined in terms of near-real-time execution, inference accuracy, energy consumption, memory footprint, etc.

6.1. Binary offloading

Binary offloading based solutions employ the device-only and server-only schemes to execute the DNN inference task. Sun et al. (2019) optimally selected an ES from a set of candidate ESs to offload the AI task obtained from end devices, considering delays (transmission and processing) and service accuracy. These AI tasks employs a pre-trained CNN models deployed on the fog computing platform for efficient abnormal event detection in machining process optimization (Liang et al., 2021b). An edge-cloud collaborative deployment was envisioned by Zhang et al. (2021a) for factory automation in performing acoustics defect detection in the beverage packing industry. They categorized the ML tasks into feature extraction (obtained at the edge node), model training (at cloud resources), and model execution (edge node to serve the real-time inference). Besides, an identical designing approach was adopted by Benbarrad et al. (2021) for an edge platform for the use-case of DNN-based intelligent product inspection.

Discussion: In device-only schemes, executing the resource-hungry DL model at the resource-scarce nodes is incompetent to fulfill the stringent computational and storage needs, which can possibly lead to an increase in the DNN inference time. However, these schemes ensure the data privacy, enabling their adoption for data privacy sensitive applications such as connected healths. In contrast, server-only schemes imposes the communication overhead and raise data-privacy concerns. These scheme can be utilized in applications requiring high computational power, such as real-time traffic analytics, with a risk factor of data privacy violation. Hence, DNN inference execution using device-only and server-only approaches results into increasing execution time due to scarce computing capacity and higher communication cost due to massive data transmission overhead from device to server. Furthermore, these solutions limits the collaborative execution of DNN inference task that reduces the efficient utilization of the available computing resources. The next section discusses the various solutions integrated with binary-offloading solutions to reduce the execution time of the DNN inference task.

[1] The keywords DNN inference task and DL inference task are used interchangeably throughout the article.

Table 2
Summary of edge infrastructure deployment solutions.

Reference	Scheme	EDs No.	EDs type	Delay	Energy consumption	Load balancing	Resource utilization	Varying traffic	IoT-ED Communication	Fault tolerance	Performance evaluation
Meng et al. (2017)	Iterative	Fixed	Cloudlet	Yes	No	No	No	No	One-hop	No	–
Jia et al. (2017)	HAF, DBC	Fixed	Cloudlet	Yes	No	Yes	Yes	No	Multi-hop	No	–
Zeng et al. (2018)	SA, GDS	Variable	ES	Yes	No	No	No	No	One-hop	No	–
Li and Wang (2018)	PSO	Fixed	ES	Yes	Yes	No	No	No	One-hop	No	10% low energy consumption and 15% efficient resource utilization
Li et al. (2018c)	k-means	Fixed	ES	Yes	No	No	No	No	One-hop	No	–
Yang et al. (2019)	Benders decomposition	Fixed	ES	Yes	Yes	No	No	No	One-hop	No	Attains near-to-optimal energy consumption performance compared with two benchmark heuristics
Wang et al. (2019)	MIP	Fixed	ES	Yes	No	Yes	Yes	No	One-hop	No	–
Fan and Ansari (2019)	Lagrangian heuristic	Fixed	ES	Yes	No	No	No	No	One-hop	No	–
Wang et al. (2020a)	BDECS	Fixed	Cloudlet	Yes	No	No	No	No	One-hop	Yes	–
Guo et al. (2020)	k-means, MIQP	Fixed	EC	Yes	No	Yes	Yes	No	One-hop	No	–
Jiang et al. (2020)	Improved k-means	Fixed	ECN	Yes	No	No	No	No	one-hop	No	–
Kasi et al. (2020)	Meta-heuristics	Fixed	ES	Yes	No	Yes	No	No	One-hop	No	–
Cao et al. (2020)	ILP, Game theory	Fixed	ES	Yes	No	Yes	Yes	Yes	One-hop	No	Attains 47.37% reduction in system response time
Chin et al. (2020)	k-clustering	Fixed	ES	Yes	No	Yes	Yes	No	Multi-hop	No	–
Lähderanta et al. (2021)	PACK	Fixed	ES	Yes	No	Yes	No	No	One-hop	No	–
Wang et al. (2020c)	DDE	Fixed	ES	Yes	No	Yes	Yes	No	One-hop	No	–
Chang et al. (2021a)	Heuristic	Fixed	ES	Yes	No	No	No	No	One-hop	No	–
Li et al. (2021)	PSO-based	Fixed	ES	Yes	Yes	Yes	Yes	No	One-hop	No	–
Wang et al. (2022b)	BGWGS	Fixed	ES	No	No	Yes	Yes	No	One-hop	Yes	Saves 10.97% of the total load and an average of 16.15% time
Li et al. (2022a)	Improved k-means	Fixed	MES	Yes	Yes	No	No	No	One-hop	No	Reduces average completion time by 15.7%, power consumption by 22%
Chen et al. (2022a)	QIP, TAKG	Fixed	ES	Yes	No	Yes	Yes	No	One-hop	No	–
Mazloomi et al. (2022)	RL	Variable	ES	Yes	No	Yes	Yes	Yes	One-hop	No	–
Ling et al. (2022)	GCN, PSO	Fixed	ES	Yes	Yes	Yes	Yes	Yes	One-hop	No	Reduces edge servers' total cost by 23.98%
Gauttam et al. (2022c)	Greedy	Fixed	ECN	Yes	Yes	Yes	Yes	No	multi-hop	No	Reduces IoT nodes delay by 23%–64%, 20%–66%, and 35%–73% on periodic, event-based, and query-based data traffic scenarios, respectively, under various network settings
Gauttam et al. (2022b)	Greedy	Fixed	ECN	Yes	Yes	Yes	Yes	Yes	multi-hop	No	Reduces the average delay per IoT node by 10%–53% and achieves 25%–68% more balanced data traffic distribution among ECNs

(*continued on next page*)

Table 2 (*continued*).

Chen et al. (2023b)	Heuristic	Fixed	ES	Yes	No	Yes	Yes	Yes	One-hop	No	–
Jiang et al. (2023)	DRL	Fixed	ES	Yes	No	Yes	Yes	Yes	One-hop	No	Outperform comparison algorithms by 13.20% to 61.84% and 23.09% to 66.32%
Asghari et al. (2023b)	Red deer, RL	Fixed	ES	Yes	No	Yes	Yes	Yes	One-hop	No	–
Asghari et al. (2023a)	BOA, COA	Fixed	ES	Yes	Yes	No	No	No	One-hop	No	–
Mehta and Verma (2024)	kMeans++, WHA	Fixed	ES	Yes	Yes	No	No	No	Multi-hop	No	Minimizes the overall network energy consumption and end-to-end delay by 60% and 35%
Tiwari et al. (2024)	PSO	Fixed	ES	No	Yes	Yes	Yes	No	One-hop	No	Improves workload balance by 40.79% and reduces energy consumption by 44.68%
Vali et al. (2024)	Greedy	Variable	ES	No	Yes	Yes	Yes	No	One-hop	No	Obtains 10% average improvement in optimizing network traffic
Asghari et al. (2024)	TSR	Fixed	ES	Yes	Yes	No	No	No	One-hop	No	Reduces energy consumption by 13% and latency by 16%
Ghasemzadeh et al. (2024)	NSGA-II	Fixed	ES	Yes	No	Yes	Yes	No	One-hop	No	Attains 8.79%–9.19% reduction in latency

6.1.1. DNN inference accelerator in binary offloading

Deploying DNNs with high-performance efficiency on edge devices is challenging due to their high computational and space complication. In this context, there are three major research dimensions for achieving on-device computation and storage, namely pruning (network weights, neurons, channels, filters, etc.), quantization, knowledge distillation, and their hybrid combinations. These approaches reduce the size and complexity of the DL model, therefore, turns them efficient for edge devices.

- ***Pruning***: Numerous high-performing DNN models utilize extensive neurons and synaptic connections, resulting in substantial computational and storage complexity. In contrast, an abundance of these are redundant, wherein eliminating them significantly reduces computation and space complexity without affecting the accuracy (Liang et al., 2021a; Cheng et al., 2024). The pruning technique determines the redundant connections or weights, neurons, channels, or filters in a DNN model and eliminates them to make the DNN model more compact (He et al., 2017; Luo et al., 2017). Le et al. presented a brainyEdge (Le et al., 2023) mechanism that utilizes weight pruning at each layer to the deployment phase of the feedforward neural network in an automated surface inspection system on an assembly line. Similarly, Zhu et al. (2020) revised the DenseNet architecture for automatic defect detection at the resource-limited edge by removing redundant connections in the DL model through pruning. Ashiquzzaman et al. (2020) applied filter pruning based on the L1 normalization in hand gesture recognition using CNN. It compresses the CNN model by 1.05% and 50% less computation while maintaining the same accuracy of 98%, resulting in lightweight mobile edge-based server applications. These mechanisms retrains the unpruned parameters for some iterations to reduce loss incurred from pruning. The layerwise filter pruning mandates computationally extensive sensitivity analysis and leads to the accumulation of pruning errors, directing performance reduction of pruned DNNs. Naveen et al. (2021) discussed a low-latency DNN inference model for a distributed edge environment. The work formulated latency depreciation as a pruning problem and utilized weight pruning followed by finetuning to acquire an optimized DL model to reduce inference latency. These pruning procedures are also extended for ensemble and semi-supervised learning techniques such as EnSeMUp in quality prediction tasks (Nain et al., 2024a). Yu et al. addressed it through EasiEdge (Yu et al., 2020), a global pruning method that quantifies the impact of filter removal by information gain (IG). Pruning 80% of VGG-16 filters reduces inference latency on Jetson TX2 CPU from 76.85 to 8.01 ms with only a 0.22% accuracy drop. In addition, CroApp (Jia et al., 2022) optimizes the CNN model's resources through channel pruning and achieves an overall resource optimization by training a pool of shareable models established on these pruned models.
- ***Early exit***: A branchy network architecture is extended to have supplementary side branch classifiers (Teerapittayanon et al., 2016). Li et al. (2018a) introduced DeepIns with the concept of early exit (branches) in the DNN to leverage the early inference at the edge devices. The edge nodes extract lower-level features and infer the output therein. The intermediate values are forwarded for the entire CNN inference at remote cloud servers only when the output at the edge cannot meet the requirements. This reduces the communication cost and the inference dependency on the cloud. The other authors extended this concept by incorporating multi-agents in the branchy DNNs to ensure production process optimization (Lin et al., 2020). The optimal selection of the early exit point of the DNN model in edge-cloud architecture also plays a significant role. Therefore, Wang et al. (2023c) proposed a segmentation mechanism for Branchy DNNs (DNNs with early exit feature) to determine the type and position of the fault in printed circuit boards. Jiao et al. (2024) discussed a CNN-based mechanism to accurately identify the location of litchi fruit among the collected images. It utilizes the channel pruning, layer pruning, and knowledge distillation to reduce model size.
- ***Quantization***: Quantization is another compression mechanism that represents the weights, gradients, and activations of the DNN model through fewer fixed-point numbers instead of 32-bit floating-point representation (Micikevicius et al., 2017; Han et al., 2015). It includes arithmetic operations that consume less power and resources, significantly reducing hardware complexity. TIP4.0 is a predictive maintenance platform that includes

quantization to reduce the DNN model size and performs inference four times faster than the uncompressed model on the CPU (Resende et al., 2021). Gil et al. (2021) utilized an effective quantization-aware DNN pruning method that runs on small and cost-effective industrial devices. The proposed UVNQ approach delivered a pruning procedure to be applied for bit precision before performing the successive quantization. To provide a hardware prototype, the data quantization module can be implanted onto edge AI chips that are adaptive and low-power consumption necessitating mechanism (Li et al., 2022c). The researchers validated the effectiveness of their algorithm and hardware modules for precise navigation and accurate classification for autonomous vehicles. Balaskas et al. (2024) described an energy-efficient inference solution for implanted DNN accelerators to cater to the demand for executing sophisticated DNN-based inferences on resource-scarce embedded devices. To this end, an automated framework is devised that jointly utilizes the pruning and quantization schemes to attain hardware-aware DNN compression in an energy-efficient manner. Moreover, RL is employed for identifying the pruning-quantization composition to obtain a trade-off between energy consumption and inference prediction accuracy (Yang et al., 2023). Rakib et al. (2024) employed an Int-8 quantization lightweight Q-CNN based solution to detect plant disease.

- ***Knowledge Distillation (KD)***: It is a procedure in which the knowledge from a large or teacher model is transferred to smaller ones (Gou et al., 2021). In this method, a lightweight and efficient smaller student model is trained specifically for EDs using a large teacher so that the small model learns the same information. Fang et al. (2020) demonstrated an on-demand DNN-based inference methods for on-premise EDs called EdgeKE. It utilizes the benefits of DNN compression based on KD (to improve accuracy and speed), further acceleration through early exit (to provide flexible choices of latency and accuracy). Others performed DL-based fault diagnosis on embedded systems in real-time using vibration signals by training a lightweight student network using the MobileNet V3 model (He et al., 2023). It reduced computation cost by 170 times with an overall accuracy rate of 94.37%, and run time was diminished by 34%. Putrada et al. (2023) proposed an edge smart lighting (EdgeSL) architecture for real-time smart lighting control in which KD-based compression methods are introduced for the k-nearest neighbor (KNN) model called DistilKNN. Indeed, they claimed it had achieved better performance than pruning and quantization. Similarly, the larger teacher model is trained in the cloud and used for secondary verification of alarm system warnings only for higher-level warning states in the smart factory. The student model at the edge performs the primary verification of most inputs (Oh et al., 2022).

Discussion: The above discussion comprises the various solutions to obtained a compressed version of DNN, whose resource requirements can be adapted by resource poor devices. HowHowever, DNN compression directly affects the model performance (such as accuracy loss), which may hamper the performance of DL-based IoT applications where model performance is essential (such as quality inspection, medical imaging, etc.). The schemes such as knowledge distillation or iterative pruning imposes the additional training overhead. The compression schemes also face deployment challenges due to the requirement of specific hardware and libraries for execution. The next section explores the partial computation offloading based solution to address the issues of binary-offloading-based DNN inference execution.

6.2. Partial offloading

The partial offloading-based solution decomposes a task into smaller subtasks and distributes them to the underlying computing infrastructure for execution. DNN splitting is a research domain enabling end-edge synergic implementation of DNN inference tasks. The synergy among the end and edge devices is named collaborative computing, which caters to near-real-time requirements (Gauttam et al., 2023). The splitting point needs to be determined in these approaches, which are based on partial computational offloading. This is the point at which the DNN layers are partitioned among collaborative devices (such as ECN and ES) and relieve the computational burdens at these edge devices. Besides, the ECN sends the intermediate outcomes from the DNN layers (layers executing at the ECN) to the ES. This is achieved considering the transmission of smaller sizes data (intermediate outcomes from ECN) instead of the actual raw data, decreasing data transmission delays. DNN splitting improves overall task execution time by optimizing computation and communication resources. This section explores the various SL-based DNN inference task execution solutions.

Kang et al. (2017) presented a DNN model partitioning mechanism between a mobile edge device and a cloud server in order to diminish latency or energy consumption. This partitioning method chooses the best splitting point depending on DNN profiling parameters (which include per-layer execution time for both device and cloud, as well as per-layer output size) and network bandwidth. After partitioning point selection, the model layers before this point are executed on mobile device and remaining on the cloud. The DNN partitioning can be employed with a right-sizing mechanism to maintain the trade-off between inference latency and prediction performance (Li et al., 2019). Zhou et al. (2019) developed a DL model partitioning mechanism utilizing dynamic programming within a multi-level IoT architecture, aiming to diminish response time. This mechanism divides the DNN layers among devices at different levels, taking into account network bandwidth and computing resources. It aligns with the framework presented in Kang et al. (2017) for device-edge, device-cloud, and edge-cloud model splitting scenarios. Boomerang is developed as an edge intelligence framework that provides on-demand cooperative DNN inference under the resource-constrained IIoT environment, executing model inference with improved latency and accuracy (Zeng et al., 2019). The early exit technique employed in Boomerang reduce the DNN inference latency while ensuring inference accuracy. Dividing the DNN computation between the IoT devices and the ES leverages hybrid computing resources and lower the model inference time. Shan et al. (2020) designed a framework named Cogent to accelerate the DNN inference task. The framework employs an RL-based solution to predict the DNN splitting and pruning to minimize the inference latency based on hardware conditions and system configuration. Saguil and Azim (2020) developed a framework that enables the creation of several DNNs and chains them together. It utilizes the concept of DNN right sizing and splitting to reduce the inference latency. The splitting points in each model are determined by discovering the areas with the largest differences in consecutive layer execution time.

The coEdge (Hu et al., 2020) framework exploits the capabilities of edge and cloud in a collaborative manner to reduce the DNN inference latency. The authors formulate the DNN splitting problem as a latency minimization problem and devise an approximate solution to achieve the optimal DNN split point in polynomial time. Some work, such as Matsubara et al. (2020), extended the DNN splitting solutions to an efficient DNN inference acceleration mechanism that integrates the characteristics of DNN splitting and Knowledge Distillation. The scheme first splits the DNN into front and back-end layers to reduce the computational burden on the end device. Further, it utilizes KD to compress the front-end DNN to improve the DNN inference performance.

DeepWear, a DL framework for wearable devices, is proposed by Xu et al. (2019) to improve DNN inference task execution performances by partitioning the computation between the wearable and paired handheld devices. It considers DNN profiling at wearable (for example, watches, etc.), handheld device, and bandwidth for partitioning while minimizing task execution latency and energy consumption. Chen et al. (2021a) introduced a computational offloading scheme called DNN-off for edge-enabled IoT systems employing DNN models. It employs

a Random Forest (RF) model to obtain the DNN profiling and partitioned the layers between device-edge and device-cloud based on the RF-based profiling outcomes and available bandwidth to reduce the total response time. Li et al. (2018b) examined a DL computational offloading method for an edge-enabled IoT scenario wherein a method is deployed to split the DNN model in edge-cloud collaboration. They attained the objective of maximizing the tasks deployed at the edge while assuring the QoS necessities. Similar work for DNN-IoT applications discussed in Chen et al. (2021a) delivered a DNN splitting scheme for offloading computational burden while minimizing the task response time. Assuming a DNN task combined many sub-tasks, they were executed sequentially to preserve the layerwise sequence of DNN model configurations. They embraced the Shortest Job First (SJF) based greedy method for scheduling the computing sub-task in edge-cloud collaboration, reducing response time. Identically, Shi et al. (2021) employed greedy strategy-based DNN partitioning in a device-edge collaboration for minimizing task execution latency in multi-task settings. Yun et al. (2022) designed a memory-aware DNN inference acceleration solution for memory and delay-constrained wireless IoT networks. The solution utilizes KD to construct a smaller student DNN model from a large teacher DNN to cope with the constrained memory of IoT devices. It consists of a dynamic DNN selection-based collaborative DNN inference that optimizes the DNN (student/teacher), the partitioning point for the preferred DNN, and the intermediate data transmission rate.

Xue et al. (2021) also designed an efficient offloading scheme for DNN inference acceleration in a device-edge-cloud cooperative setting to realize the real-time DNN query. It utilizes a PSO and GA empowered hybrid solution and a layer merge uploading algorithm to obtain DNN migration and partition plan and maintain efficient query performance. enabling Distributed DNN (eDDNN) (Huang et al., 2021), are designed for distributed DNN inference schemes in an environment of heterogeneous end devices. It leverages a cross-platform WebRTC protocol that splits tasks horizontally, allowing the DNN model to be executed nearly independently on end devices by sharing a DNN dependency table. The solution also comprises an RL-based dynamic task allocation algorithm to efficiently utilize the computing power of the collaborative field-devices that reduce the overall latency and lower the computational burden on ESs. Chen et al. (2021b) designed an energy-efficient offloading technique using a self-adaptive PSO algorithm based on the GA operators to divide the DNN layers while reducing system energy consumption and ensuring deadline constraints.

The task partitioning and offloading schemes can be employed collectively for a DNN-task-enabled MEC network comprising a single server and numerous mobile devices (MDs) to reduce inference delay and energy consumption (Gao et al., 2021). The optimal DNN offloading must be attained with load balancing, and therefore, Xu et al. (2022) described a distributed auction algorithm that attained it them in polynomial time to reduce the inference latency and efficient utilization of edge resources. Yang and Wei (2022) designed a coalition formation method to achieve privacy and energy-aware split learning in edge camera networks to infer DL tasks on the computationally poor edge-IoT system. The solution performs user association decisions, resource allocation for edge cameras, and device configuration with DL model parameter selection. It utilizes a coalition formation game-based scheme to solve the association problem between edge nodes and cameras. Further, a split DL solution is adopted for flexible allocation of computing and communication resources. Thus, the coalition formation and split DL jointly maximize the multi-view detection, privacy preference, and energy savings of cameras. Liu et al. (2023b) jointly explored a model partitioning solution with a mobility-aware computation offloading solution among the cooperative devices by considering the characteristics of Vehicular EC (VEC). These characteristics are higher mobility, dynamic network topology, and irregular wireless connections to reduce DNN inference latency and ensure reliability. Nature-inspired optimization schemes such as Improved Particle Swarm Genetic mechanism (Li et al., 2023a) determined an optimized computational offloading strategy through DNN partitioning to reduce the overall task execution latency in VEC. Feltin et al. (2023) formulated the DNN partitioning problem as a MINLP. It employs a branch and bound solver to partition the given DNN model to maximize the throughput of DNN inference tasks and cater to the latency and privacy-sensitive requirements of DL-based IoT applications. Dynamic Adaptive DNN Surgery mechanism enabled the split DNN to be executed in edge and cloud synergy while reducing the data transmission (Liang et al., 2023) and optimally splitting the model under varying network scenarios. In addition, they analyzed the DNN splitting in a cost-constrained scenario such that the cloud consists of limited resources for the DNN inference task. It employed DNN Surgery Light and Heavy to attain optimal DNN partition under lightly and heavily loaded networking scenarios. DNN splitting is merged with the resource allocation optimization approaches to reduce the execution of model inference tasks in the edge or edge-cloud IoT scenarios (Fan et al., 2023). The DL model splitting is modeled as MINLP and divided into two smaller subproblems due to the higher computational cost. The first subproblem is solved using graph-based device clustering for reducing the data transmission delay between device and base station. Next, the second subproblem utilizes DNN splitting and resource allocation mechanisms jointly to minimize computing and transmission delay between the base stations and the cloud. Su et al. (2022) investigate a joint optimization problem in a hierarchical energy-constrained edge–cloud environment for the minimization of the average end-to-end delay of various DL tasks while ensuring that the energy consumption of the ES and cloud remains within the allocated energy budgets. The authors achieved it through an RL-based solution to dynamically partition the DNN by monitoring the environment. Dong et al. (2022) presented an investigation, concluding that the performance of a DNN inference task is influenced collectively by exit point determination, DNN partitioning, and resource allocation in edge AI scenario. It employs a bidirectional dynamic programming empowered exit point selection algorithm. it also employs a DRL-based scheme that learned from offloading decisions and determined optimal resource allocation and DNN splitting point. Wang et al. (2023a) designed a dynamic optimization approach for DNN splitting under the scenarios of fluctuating wireless channel(s) and random DNN task arrival. They addressed the challenges of varying wireless channel conditions and DNN tasks in multiuser MEC systems and minimized the average latency of inference tasks. The dynamic DNN task workload at end devices and MEC server is modeled using a local CPU and MEC server workload queue. It dynamically determines the partitioning of the local DNN task at the wireless device using a parametric online Q-learning algorithm by considering the instantaneous status of channel conditions, end device status, and status of MEC server, respectively, to reduce the average latency of the DNN inference tasks.

Jung and Lee (2023) developed a framework for low-latency DNN inference task computation over distributed computing platforms to reduce inference latency. It utilizes routing in the layered graph model to enable the selection of nodes for computation and paths for data transfer. Others proposed a wireless channel adaptive DNN partition framework (Lee et al., 2023) in energy and memory constraints edge environment, minimizing the inference latency. The framework employs a two-stage DNN splitting, where DNN splitting points for the deployment are decided offline by considering the mean of transmission latency. Next, it adaptively adjusts the deployed DNN model on the edge-cloud synergy in an online fashion based on the instantaneous channel conditions. Dong et al. (2023) discussed a decentralized DRL scheme to accelerate the DNN inference by collaborating resource-constrained IoT nodes and mobile edge computing. They optimized distributed task offloading for intelligent edge or end devices to tackle the computation and communication challenges of DNN inference execution using a multi-agent RL-based solution that selects an early exit point in the DL model for balancing accuracy and latency. Such

solutions are diversified in the applications of resource-constrained Internet of Vision Things (Xu et al., 2023b) for edge-device collaborative inference to accomplish DNN-based inference efficiently. They collectively adopted early exit architectures with resource allocation schemes and modeled this joint problem to speed up the DNN inference while ensuring the desired inference accuracy. They employed a MDP and a deep deterministic policy gradient approach to cater to the above-mentioned issues in an edge-device collaborative set-up.

A novel approach, known as NNFacet (Chen et al., 2023a), is a DNN splitting framework to conduct a DNN-based classification task on N simultaneous battery-resource-constrained sensors with reduced latency and energy consumption. NNFacet divides a large multi-class DL model into several smaller models wherein each small model is deployed on a resource-constrained smart sensor that learns a subset of the classes. The final inference is generated by fusing the inference of sensor nodes produced by their submodel. Duan and Wu (2023) presented a solution which collectively optimize DNN partition and pipelines the scheduling for various DNN inference jobs for minimizing the latency. The work designed a near-optimal splitting and scheduling strategies based on binary search, and idea of topology sort for chain-structure extended them for general-structure DNNs. Wu et al. (2023) discussed a splitting mechanism for DNN consisting of Directed Acyclic Graph (DAG) topology to minimize the inference latency. The solution employs a topological sort-based mechanism to convert DAG-topology DNNs into chain-topology DNNs and applies a greedy approach to partition and offload chain DNNs. Considering a heterogeneous dynamic IoT environment, Samikwa et al. (2023) discussed a distributed micro-split DL to attain the optimized trade-off between latency and energy consumption. The solution comprises a heuristic-based mechanism that iteratively performs vertical and horizontal DNN partitioning to carry out DNN execution in a distributed manner in the network of IoT nodes with diversified computing abilities and varying network situations without compromising the DNN model performance. Lim et al. (2024) addressed the combined optimized dynamic DNN model splitting, processing, and network resources to enhance the quality of experience (QoE) for DL-based IoT applications in the edge computing paradigm. The authors designed an online and dynamic DNN splitting mechanism to decrease the energy consumption and average end-to-end latency of the DNN inference tasks. Consequently, Li et al. (2024a) studied it further for fine-grained model partitioning among the collaboration of ES-IoT assets to address the challenges of the DNN inference execution task at a heterogeneous edge-IoT environment. The DNN splitting problem is formulated as the MDP, for maximizing the long-term ignored cumulative reward of distributed DNN inference tasks. The authors utilized a multi-task learning-based asynchronous advantage actor-critic approach to finding the optimal DNN splitting policy to reduce DNN inference execution delay.

Li and Bi (2024) examined an RL-based efficient DNN splitting solution to attain device-edge collaborative DNN inference for multi-user wireless sensing to enable the accurate detection of various human activities through the analysis of various radio signal variations of sensor devices in near-real time. PArtNNer (Ghosh et al., 2024), is a system that enables distributed DNN inference in a single ED-cloud scenario to reduce DNN inference latency. The scheme determines the decision to drive the inference task exclusively on edge, entirely in cloud, or in edge-cloud collaboration by analyzing the influence of temporally altering underlying operating conditions, computing, and communication architecture, respectively. Ko et al. (2024) designed a Two-Phase Split Computing (TPSC) framework to reduce the inference latency of DNN inference tasks. The framework utilizes inter-layer and intra-layer splitting to attain the same, where inter-layer splitting performs the layerwise splitting and intra-layer splitting divides the computing task of a layer among the various nodes at the same level. Qi et al. (2024) discussed a Multi-Compression Scale DNN Inference Acceleration (MCIA) solution in end-edge-cloud collaboration in the multi-user multi-task environment to cope with the scenarios of distinct requirements of DNN inference services (for instance, highly accurate or latency-sensitive tasks). It deploys multiple versions of the DNN models with different compression scales at field devices and ED, and utilized a DRL algorithm for adaptive and collaborative inferences with optimal resource allocation.

Discussion: The above discussion involves the investigation and analysis of various partial-offloading-based DNN inference execution solutions (such as SL and its integration with other DNN inference accelerator (based on application requirements) schemes) to minimize the execution cost of the DNN inference task. The SL-based schemes attains the optimized utilization of the available computing and storage resource. These solutions transmit the intermediate data among collaborative devices, which addresses the data-privacy. The core SL-based solution (i.e. without any additional DNN accelerator) did not alter the DNN architecture that mitigates the model performance degradation. It demonstrates that these solutions are suitable for scenarios where maintaining model accuracy is critical, for example, quality inspection in smart manufacturing. However, model splitting introduce an additional overhead, which may degrades the performance of real-time applications, initiating the needs for lightweight model splitting schemes. The summary of existing edge inference schemes is presented in Table 3.

7. Model training optimization layer

This section discusses the challenges and respective solutions in the context of the third layer (model training optimization) of DEMMA. DNN training task[2] comprises a DNN model and training data set. It aims to optimize the randomly initialized weights of a given DNN by minimizing a loss function using optimization algorithms (such as gradient descent) and backpropagation mechanism. Edge training (also referred to as edge learning) employs the solution that enables the training of DNN models on edge infrastructure. It reduces the dependency on the centralized cloud server for DNN training tasks. However, the limited resources (computing and storage) at the edge impose significant challenges for the successful execution of resource-hungry DNN training at the edge. This section presents an investigation and discussion of the contemporary literature, comprising the solutions utilized to address the challenges of DNN training task at the network edge. Fig. 8 depicts the categorization of DNN training task execution solutions in Edge-AI paradigm. The mechanisms broadly employs the solo (on-device/edge) and distributed/collaborative learning approaches. The DNN training is distributed among numerous EDs in split or distributed learning, whereas the on-device/edge learning executes the DNN training on individual ED. Further, these schemes integrates the DNN training accelerators such as model compression, KD, quantization, etc., for performance enhancement.

The choice of a DNN training execution scheme is affected by application requirements (e.g., convergence time, accuracy, and reliability) and available computational, storage, and network resources. For instance, in an autonomous vehicle driving scenario, a vehicle-only training mechanism (solo training) executes the model updates locally using onboard resources. It preserves data privacy and eliminates network dependency, but requires significant computing power and storage to achieve the desired model accuracy within an acceptable time frame. In contrast, an edge-only training scheme can support the training of complex models, but it increases communication overhead due to raw data transfer from vehicles to edge servers and may raise privacy concerns. Distributed training schemes (such as federated learning, split learning, etc.) utilize the resources of vehicles and edge devices. These approaches maintain model quality and data privacy by sharing only model updates or intermediate representations, though

[2] DNN training/learning task and DL model training/learning are used interchangeably throughout the article.

Table 3
Summary of Edge-AI empowered DNN inference solutions.

Reference	Offloading	DNN type	DNN task	Inference accelerator	Inference delay	Inference energy	Device synergy	Varying resources	Memory footprint	Performance evaluation
Sun et al. (2019)	Binary	CNN	Multiple	None	Yes	–	None	No	No	–
Liang et al. (2021b)	Binary	CNN	Single	None	Yes	–	None	No	No	70.26% reduction in the bandwidth requirement and 47.0270.26% reduction in data transfer time
Zhang et al. (2021a)	Binary	ML	Single	None	Yes	–	None	No	No	Defect detection time for one bottle is less than 2.061 s
Benbarrad et al. (2021)	Binary	CNN	Single	None	Yes	–	None	No	No	–
Le et al. (2023)	Binary	FNN	Single	Pruning	Yes	No	None	No	Yes	–
Zhu et al. (2020)	Binary	CNN	Single	Pruning	Yes	No	None	No	Yes	Attains 32% reduction in latency
Ashiquzzaman et al. (2020)	Binary	CNN	Single	Pruning	Yes	No	None	No	Yes	–
Naveen et al. (2021)	Binary	CNN	Single	Pruning	Yes	No	None	No	Yes	Reduces communication size by ≈14.4% and inference latency by ≈16%
Li et al. (2018a)	Binary	CNN	Single	Early-exit	Yes	No	None	None	No	–
Lin et al. (2020)	Binary	DNN	Single	Early-exit	Yes	No	None	No	Yes	–
Wang et al. (2023c)	Binary	DNN	Single	Early-exit	Yes	No	None	No	Yes	–
Jiao et al. (2024)	Binary	CNN	Single	Pruning, KD	Yes	No	None	No	Yes	Exhibited a 1.8-fold increase in speed compared to the unprocessed model
Resende et al. (2021)	Binary	ML	Single	Quantization	Yes	No	None	No	Yes	–
Gil et al. (2021)	Binary	DNN	Single	Quantization	Yes	No	None	No	Yes	–
Li et al. (2022c)	Binary	CNN	Single	Quantization	Yes	Yes	None	No	Yes	–
Balaskas et al. (2024)	Binary	CNN	Single	Pruining, Quantization	Yes	Yes	None	No	Yes	39% average energy reduction for 1.7% average accuracy loss
Rakib et al. (2024)	Binary	CNN	Single	Quantization	Yes	Yes	None	No	Yes	–
Fang et al. (2020)	Binary	CNN	Single	KD, early-exit	Yes	No	None	No	Yes	Speedup ratio is up to 3.30× under 4.84% accuracy loss
He et al. (2023)	Binary	CNN	Single	KD	Yes	No	None	Yes	Yes	–
Putrada et al. (2023)	Binary	KNN	Single	KD	Yes	No	None	No	Yes	–
Oh et al. (2022)	Binary	CNN, RNN	Single	KD	Yes	No	None	No	Yes	–
Kang et al. (2017)	Partial	CNN, ANN	Single	Split	Yes	–	Device-Cloud	Yes	No	Improves end-to-end latency by 3.1X× and reduces mobile energy consumption by 59.5%
Li et al. (2019)	Partial	CNN	Single	Split, early-exit	Yes	No	Device-Cloud	Yes	No	–
Zeng et al. (2019)	Partial	CNN	Single	Split, early-exit	Yes	No	Device-ES	Yes	No	–
Zhou et al. (2019)	Partial	CNN	Single	Split	Yes	–	Device-ES-Cloud	Yes	No	–
Shan et al. (2020)	Partial	CNN	Single	Split, Pruning	Yes	No	Device-ES	Yes	No	Obtains infrence accelerate up to 8.89× without loss of accuracy of more than 7%
Saguil and Azim (2020)	Partial	CNN	Single	Split, early-exit	Yes	No	Device-ED	No	No	Reduces messages transfer cost by up to 97% with a 3% accuracy reduction
Hu et al. (2020)	Partial	CNN	Single	Split	Yes	No	Device-ED	No	No	–
Matsubara et al. (2020)	Partial	CNN	Single	Split, KD	Yes	No	Device-ED	Yes	No	–
Xu et al. (2019)	Partial	CNN, RNN	Single	Split	Yes	–	Device-ES	Yes	No	Introduce 5.08X–23.0X execution speedup, as well as 53.5–85.5 percent energy saving
Chen et al. (2021a)	Partial	CNN	Single	Split	Yes	–	Device-ES	No	No	Reduces response time by 12.4–66.6%
Chen et al. (2021b)	Partial	CNN	Single	Split	Yes	Yes	ES-Cloud	No	No	–
Yun et al. (2022)	Partial	CNN	Single	KD, Split	Yes	No	Device-ES	Yes	Yes	–
Xu et al. (2022)	Partial	CNN	Single	Split	Yes	No	Device-ES	Yes	No	Improves inference execution 1.14 to 2.62 times faster
Dong et al. (2022)	Partial	CNN	Single	Split, early-exit	Yes	No	Device-ES	Yes	No	Accelerate DNN inference by up to 13.7 times
Lee et al. (2023)	Partial	CNN	Single	Split	Yes	Yes	ES-Cloud	Yes	No	–

(*continued on next page*)

Table 3 (*continued*).

Samikwa et al. (2023)	Partial	CNN	Single	Split	Yes	Yes	Device-ED	Yes	No	Reduces the DNN inference latency and energy consumption by up to 5.2× and 6×
Chen et al. (2023a)	Partial	CNN, RNN	Single	Split	Yes	Yes	EDs	No	No	–
Li and Bi (2024)	Partial	CNN	Single	Split	Yes	Yes	Device-ED	No	No	–
Ko et al. (2024)	Partial	CNN	Single	Split	Yes	No	Device-ED	Yes	No	Reduce inference latency by 38.8%
Li et al. (2018b)	Partial	CNN	Multiple	Split	Yes	–	ES-Cloud	No	No	–
Chen et al. (2020)	Partial	CNN	Multiple	Split	Yes	–	Device-ES	No	No	–
Shi et al. (2021)	Partial	CNN	Multiple	Split	Yes	–	Device-ES	Yes	No	Reduce the system time cost by 28–30%
Huang et al. (2021)	Partial	CNN	Multiple	Split	Yes	Yes	Device-ES	No	No	Obtains the latency reduction by 2.98× and reduce mobile energy consumption by 1.8×
Xue et al. (2021)	Partial	CNN	Multiple	Split	Yes	No	Device-ES-Cloud	No	No	–
Gao et al. (2021)	Partial	CNN	Multiple	Split	Yes	Yes	Device-ED	Yes	No	–
Yang and Wei (2022)	Partial	DNN	Multiple	Split	Yes	Yes	Device-ES	Yes	No	–
Su et al. (2022)	Partial	CNN	Multiple	Split	Yes	Yes	ES-Cloud	Yes	No	–
Liu et al. (2023b)	Partial	CNN	Multiple	Split	Yes	No	Device-ES	No	No	–
Li et al. (2023a)	Partial	CNN	Multiple	Split	Yes	No	Device-ES	No	No	–
Feltin et al. (2023)	Partial	DNN	Multiple	Split	Yes	No	Device-ES	Yes	No	–
Liang et al. (2023)	Partial	DNN	Multiple	Split	Yes	No	ES-Cloud	Yes	No	Improve latency up to 6.45 times
Fan et al. (2023)	Partial	DNN	Multiple	Split	Yes	No	ES-Cloud	Yes	No	Attains 28.3% delay reduction
Wang et al. (2023a)	Partial	DNN	Multiple	Split	Yes	No	ES-Cloud	Yes	No	–
Jung and Lee (2023)	Partial	CNN	Multiple	Split	Yes	No	EDs	Yes	No	–
Dong et al. (2023)	Partial	DNN	Multiple	Split, early-exit	Yes	No	Device-EDs	Yes	No	–
Xu et al. (2023b)	Partial	CNN	Multiple	Split, early-exit	Yes	No	Device-EDs	Yes	No	–
Duan and Wu (2023)	Partial	CNN	Multiple	Split	Yes	No	Device-EDs	Yes	No	–
Wu et al. (2023)	Partial	CNN	Multiple	Split	Yes	No	EDs	Yes	No	–
Lim et al. (2024)	Partial	CNN	Multiple	Split	Yes	Yes	Device-ED	Yes	No	–
Li et al. (2024a)	Partial	CNN	Multiple	Split	Yes	Yes	Device-EDs	Yes	No	Reduces total inference delay, edge inference delay and local inference delay by 4.76%, 10.04% and 8.03%
Ghosh et al. (2024)	Partial	CNN	Multiple	Split, compression	Yes	No	Device-ED-Cloud	Yes	No	Improves end-to-end inference latency by 10.2 times to 21.1 times
Qi et al. (2024)	Partial	CNN	Multiple	Split, compression	Yes	No	Device-ED	Yes	Yes	–

they introduce additional overhead for coordinating training, managing offloading decisions, and update aggregation. Thus, the purpose of edge-based training is to conduct the DNN training tasks at the edge where the data is generated and/or collected, while catering the required performance metrics.

7.1. Solo (on device/edge) DNN learning in Edge-AI

In the context of on-device learning, ME-FEEL (Tang et al., 2022) is a novel multi-exit federated edge learning that accomplishes the DL training and model weight sharing efficiently in a resource-constrained environment. The framework divides the DL model into different sub-models of various depths and output prediction from the exits in their respective sub-model. It enables the selection of early exits and avoids training the complete model at poor computational devices, which, in turn, reduces model training latency. The performance characterization of DNN model training on various EDs (three contemporary classes of NVIDIA Jetson) by varying the device (such as storage media) and training parameters (such as batch size) is investigated in SK et al. (2022). Wang et al. (2023b) developed a training approach for Binary Neural Network (BNN) training that is low-cost and suitable for resource-constrained EDs. It is adaptive for attaining sizable memory footprint contractions at the cost of little to no accuracy drops compared to standard benchmarks. Kook et al. (2024) presented an efficient offloading architecture called joint data deepening-and-prefetching to reduce the transmission cost of high-dimensional and voluminous data from resource-constrained IoT devices to EDs for DL training. It uploads the features of data based on importance rather than raw data, which reduces the energy consumption of data transmission. Further, the pre-fetching steps determine the potential features required for future training in advance to enhance precision and mitigate energy consumption overhead.

Yu et al. (2024) compiled the pruning approach with quantization to achieve quick training and/or retraining of new tools, machines, or processes with scarce data in the industrial anomaly prediction of equipment. The model size is diminished by about 60%, making it realizable for EDs like Raspberry Pi through pruning and quantization. Sung and Han (2024) discussed a model maintenance approach in the 5G IoT network for IoT-AI applications. The approach employs the concepts of model pruning and approximate learning to maintain the

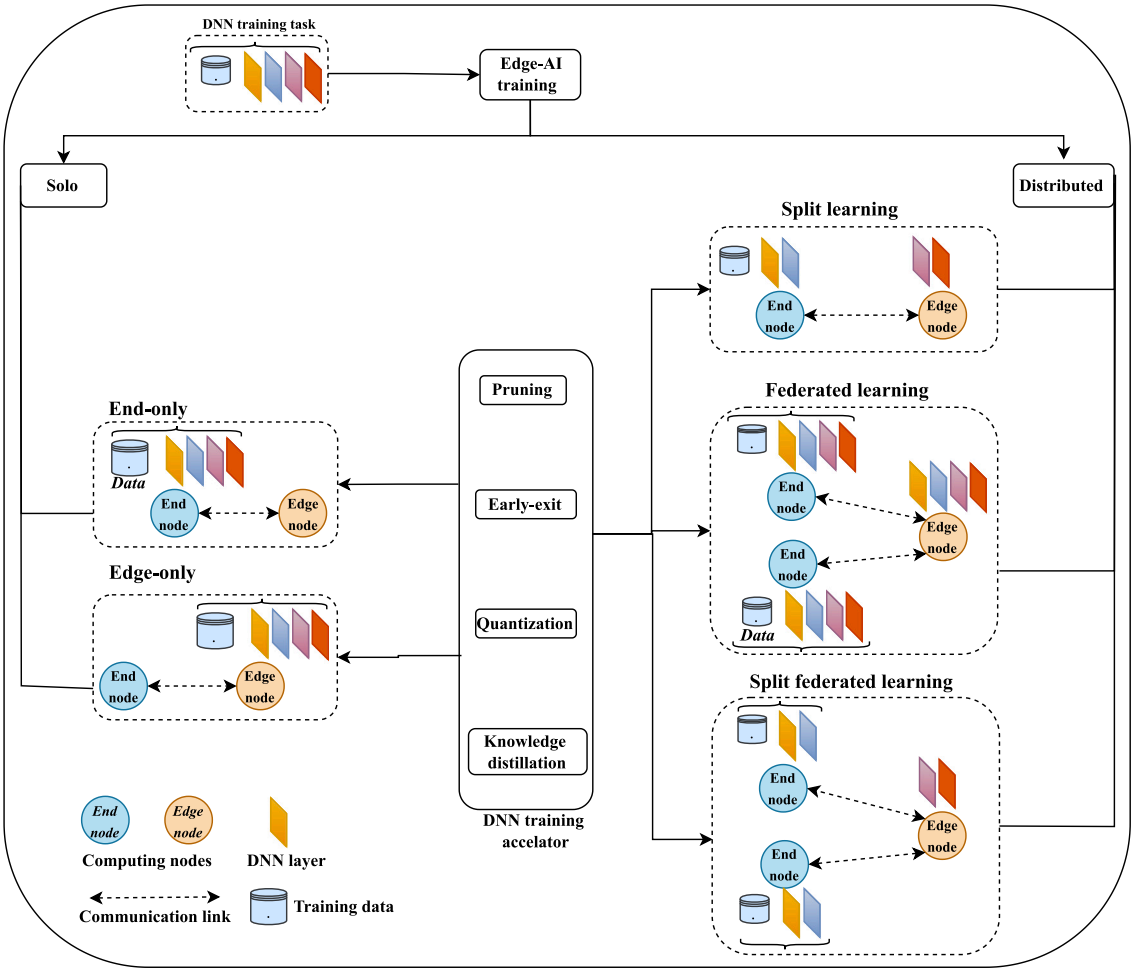


Fig. 8. Edge-AI training schemes.

trade-off between model performance and resource consumption for DNN training. A DRL scheme obtains a lightweight DNN from the full-scale DNN model to attain learning from the online data based on the above trade-off. XBNet-Based Framework (Hamad, 2025), utilizing the collective features of ML and genetic algorithms, is proposed to enhance gene-disease classification. It combines the advanced feature extraction and selection technique, which improves the classification performance.

Discussion: The DNN training procedure follows an iterative and compute intensive forward and backward prorogation. The schemes stated above illustrates the techniques to achieve on-device DNN training. The on-device DNN training schemes are a suitable for the data-privacy sensitive applications such as connected health. It is also relevant for the applications such as precision farming, comprising remote/limited bandwidth deployment. However, limited resource availability raises the concerns such as the higher training time, increased energy consumption, model performance degradation, limited scalability, overhead of compression algorithms, etc., which are addressed in next sections.

7.2. Distributed DNN learning in Edge-AI

This section discusses the distributed learning approaches, including Federated Learning (FL), Split Learning (SL), and concurrence of FL and SL to overcome the limitations of solo DNN learning schemes. The discussion also includes the acceleration methods utilized to speed up the distributed DNN training.

7.2.1. Federated learning

Data privacy is a vital concern in various scenarios, including healthcare, autonomous vehicles, smart manufacturing, and more. Hence, the data cannot be shared in the remote cloud for model training because it contains confidential or sensitive information or due to compliance and legal concerns. The efficient handling of distributed sub-datasets in the above-mentioned scenarios calls for decentralized learning solutions. In this context, a FL solution trains a local model at each edge site and aggregates their parameters at the global model at the ES or cloud without sharing the sensitive data (Wang et al., 2022a). Hence, each ED maintains the privacy of the data at each local site and prepares a global DNN model, which is updated at each ED. A FL framework is designed for the wirelessly connected swarm of UAVs, comprising a leader UAV and a set of followers UAVs (Zeng et al., 2020). Each follower UAV gathers data to train its own local DL model and forwards the revised model parameters to the leader UAV to generate the global model. Brik et al. (2022) utilized the FL to design a prediction model for resource localization of mobile resources in industrial systems. Some research, such as Li et al. (2022b), focused on optimizing the weight coefficients of clients involved in parameter aggregation. It involved utilizing an improved PSO and also embedded a method for selecting appropriate models (or the FL client) to update the global FL model. They propose a stacking model for diagnosing inter-turn short circuit faults in permanent magnet synchronous motors. Farahani and Monsefi (2023) discussed privacy-enhancing FL in the inter- and intra-organizational collaboration of IIoT data to present a holistic, decentralized architecture and validated its efficacy in an edge-enabled collaborative predictive diagnostics and maintenance case

study. In addition, Putra et al. (2023) suggested a hierarchical federated transfer learning for simultaneously learning and detecting faults in the 3D-printing process that has 24% faster training time and 45% accuracy improvement compared to traditional FL architecture.

Accelerating the FL: FL is a novel training technique that protects data privacy by building shared learning of the DL/ML model from the training data that is locally available on distributed edge nodes. However, the model update in FL can be bandwidth-consuming as it requires parameter exchanges among edge nodes in iterative fashion. The communication cost among collaborative devices (ECN/ES/Cloud) is high when updating models through FL. Tonellotto et al. (2021) proposed the models to undergo a first quantization after being updated by the edge servers, followed by a second quantization before transferring global model from cloud node to the edge servers. Zheng et al. (2021) introduced a distributed hierarchical tensor deep computation model to reduce FL's bandwidth consumption and storage requirement by reducing the DL model's parameters from high to low-dimensional tensors. In a similar work, Wang et al. (2020d) integrated KD into FL to overcome privacy issues and transmission costs in object recognition tasks in IoT scenarios. Zhang et al. (2020) designed a big-little branch architecture inspired by BranchyNet to enable efficient FL for AIoT applications. The scheme deploys the little branches of the DNN model on field or IoT devices due to the availability of limited resources and big branches on cloud servers to improve prediction accuracy. Tanghatari et al. (2022) discussed an approach to enable distributed DNN training on resource-deprived IoT devices. A heuristic approach converts the large DNN model (teacher) into the smaller model (student) based on the available resource characteristics of the IoT node. The knowledge of the teacher is conveyed to the student model using KD. The global model is obtained through the aggregation of student model parameters at ES. The scheme reduces the data transmission cost between ES and IoT devices due to the transmission of fewer parameters of the student model and attains 3.5% accuracy loss compared to cloud-based training.

Mamba Kabala et al. (2023) employ the FL approach to train the attention-based CNN for disease detection to improve productivity, reduce costs, and promote environmentally friendly crop treatment techniques. De Rango et al. (2023) designed an energy-efficient hierarchical FL approach for the EC paradigm. The hierarchical architecture-based FL scheme exploits the benefits of available computing power at various layers of edge infrastructure for data aggregation, which, in turn, reduces the data transmission cost. An edge FL solution for the network of mobile nodes under the optimization of delay, energy, and cost for maximizing the probability of prosperous receiving of the local models at ES, respectively, is presented by Pervej et al. (2023). Lian et al. (2023) merged the features of FL and model sparsification to create a distributed, privacy-preserving, and communication-efficient solution for traffic sign recognition. Djenouri et al. (2023) designed an FL framework to preserve data privacy and reduce the training cost of a GCNN to improve the performance of traffic flow prediction in a smart city scenario. Besides this, Aggarwal et al. (2024) designed a resource-efficient, lightweight, and privacy-friendly FL framework by integrating the characteristics of transfer learning and feature extraction for Internet of Agriculture Things (IoAT) scenarios. FL has applications in predicting soil-based crop recommendations (Mwawado et al., 2024). In this work, the global model is initialized on a server and distributed to the Raspberry pi nodes to train the model on the respective local dataset. Each node shares the locally trained model with the server to obtain an efficient global model. Bera et al. (2024) discussed an IoT-dew-edge-cloud collaborative latency and energy-aware FL framework for sustainable irrigation. The ESs accomplish the training of local models on respective datasets and transmit the revised parameters to a cloud server for aggregation. To reduce the per-round latency in FL (leading to a reduction of overall training time), Zhang et al. (2024a) utilized the gradient quantization approach. A DDRL scheme allocates the quantization level to maintain the trade-off between quantization error, model accuracy, and training time, respectively. Adjei-Mensah et al. (2024) introduced a novel attention-based feature extraction mechanism that enhances the image classification performance and integrates it with FL to ensure privacy-preserving model training using medical data.

7.2.2. Split learning

SL is a distributed DL framework that splits the DNN layers between the collaborative resource-constrained EDs to reduce the computational burden of EDs and preserve data privacy (Gupta and Raskar, 2018). Zhang and Xu (2021) discussed an efficient DNN partition framework for collaborative DNN model training to cater to privacy requirements. It comprises Learn to Split (L2S), an online approach that utilizes a bandit learning algorithm to optimally split the DNN layers dynamically, preserving data privacy and enhancing training performance. A reliable ML-based computational task offloading framework utilized the features of SL to ensure the data privacy and minimum data exchange between participating EDs (Bharti and McGibney, 2022). Akintoye et al. (2022) described a scalable and distributed DL mechanism by jointly employing features of data and model parallelism to speed up the training time. The authors devised a Genetic Algorithm Heuristic Resources Allocation scheme that splits the DNN model into multiple groups of layers and allocates them to available computing devices to reduce the computing cost and improve resource utilization. Samikwa et al. (2022) presented a resource-aware SL approach for distributed DNN training in an edge-IoT environment. The solution involves the underlying network and computational resources in the decision-making process of split-DNN layers among devices to minimize training time. Wang et al. (2023d) designed a privacy-preserving SL framework, Masked Split Learning (MaskSL), to facilitate sensitive image pre-training with large-scale models in device-server synergy. It utilizes the concept of masked pre-training to minimize the computation and communication overheads. The mechanism masks patches based on Fisher information to improve training performance under limited bandwidth and privacy constraints. Further, to improve the scalability and memory efficiency of large-scale DNN training on large-scale computing clusters, Li et al. (2023b) designed a hybrid parallel training framework. The framework splits the DNN model using an integrated pipelined model and data parallelism to ensure high throughput, low memory overhead, and scalability in DNN model training on large-scale clusters.

Lien et al. (2024) employs a split computing approach to reduce the overall DNN training time for next-generation smart networks to enable the support for AI applications. A multi-tier DRL scheme obtains the distribution of the DNN model between the user device and edge, considering the model accuracy, energy consumption, and training latency, respectively. A DNN partitioning and scheduling scheme in heterogeneous IoT environments is designed to reduce training cost (time) and enhance resource utilization of resource-constrained devices (Huang et al., 2024). The solution adopts a double network policy gradient-based model comprising a stage partitioning network (employs fully connected networks) and a stage scheduling network to adaptively split the DL model into sub-stages and efficiently schedule these stages (or replicas) on end devices. Identically, Al Maruf et al. (2024) discussed a mechanism for DNN model parallelism to address the issues of optimal DNN model partition determination and their distribution among embedded devices to enhance DNN training performance. The solution jointly utilizes the characteristics of data and model parallelism, as well as pipeline-based execution, to attain higher training efficiency. The framework employs two modules: (i) identifies optimal DNN split by considering the model layers, processors' computing capacities, computational needs of various DNN layers, and communication latency (ii) minimizes the model training time using pipeline-based model parallelism. The inclusion of pipeline-based execution caters to resource under-utilization of the task dependency scenarios for model parallelism, where processors rely on each other for inputs.

7.2.3. Split federated learning

In general, FL converges more quickly than SL, while SL tends to require fewer client-side resources than FL. For instance, Guo et al. (2021) designed LightFed, a novel federated edge learning system to cope with communication and data privacy issues. The framework consists of a model-splitting mechanism that enables the edge nodes to transmit only partial models. Further, a Selective Parameter Transmission Protocol (SPT) is devised to automatically detect and filter out parameter vectors with large gradients, reducing communication costs. Bharti and Mcgibney (2021) utilized the characteristics of SL to design a framework to train the DNN at resource-deprived EDs for collaborative predictive maintenance in smart manufacturing. Liu et al. (2022) discussed a distributed learning algorithm named hybrid split and federated learning (HSFL), comprising the parallel learning and structure splitting features of FL and SL to train the DNN in a distributed fashion in wireless UAV networks. It only shares model parameters that enable data privacy. The authors also designed a multi-arm-bandit-based user device selection scheme by considering device computing capability, available channel condition, and the significance of local model updates to enhance the performance of FL. Deng et al. (2022) designed a compute-communication efficient and distributed framework to execute resource-hungry FL on resource-deprived IIoT devices. It splits the DNN model, which allows the training of lower layers at IoT devices, and offloads the higher layers to the ES for training. Moreover, data heterogeneity plays a vital role in the FL's efficiency, and it degrades in the presence of non-IID data distribution. To this end, the authors devised a participating device selection scheme, which selects an IoT device (based on data distribution characteristics) among the available devices to improve the FL enactment.

Cheng et al. (2023) designed a distributed clustering-based HFSL framework for edge networks. The learning clients can create a cluster with their neighbors (or helping) clients through D2D communications to complete collaborative-FL. The framework splits the DNN model and allocates the partitioned segment to cluster members. Next, in each cluster, a transmission client is determined, which transmits the locally trained model of that cluster to the server for global model aggregation. The solution employs a game-theoretic approach, a heuristic solution, and a greedy-based convex optimization for cluster formation, DNN splitting and allocating splinted segments to cluster members, and transmission client selection, respectively. Feng et al. (2023) discussed an efficient distributed learning framework, IoTSL, using cloud–edge collaboration in IoT systems to address SL's higher communication, computation, and energy consumption. The work utilizes the fundamental concepts of SL to split the DNN model layers among the participating devices. Xu et al. (2023a) designed an SFL framework that employs the features of parallel training of FL and model splitting of SL to train an efficient global DNN model. The author formulates the problem as a joint optimization of splitting point selection and bandwidth allotment for minimizing the training latency and discusses an alternating optimization-based solution proposed for the same. Khan et al. (2023) discussed the hierarchical split FL, a novel framework that enables distributed DNN model training at resource-constrained end devices. It operates on the grouping principle, where, each group comprises a set of end devices and an ES. The DNN model is split between the ES and end devices to minimize the computation overhead on end devices. In each group, the ES aggregates the local partial models of end devices and transmits the updated model parameters back to end devices to update the model accordingly. The procedure follows iteratively for some epochs. Further, the solution produces the global model by aggregating the local model of ESs. Chen et al. (2023c) discussed a novel DNN training approach by utilizing the concept of SL to train a DNN model in the network of distributed heterogeneous devices with a fault-tolerance mechanism. The solution dynamically determines the DNN splitting points by considering the devices' real-time computing capacities.

Guo et al. (2023) described a SFL scheme that enables the DNN co-training between resource poor EDs and server to minimize the total system latency. The solution jointly optimizes the partition layer selection and bandwidth allotment in SFL, minimizing total latency. It decomposes this into two sub-problems, where the solution of the first subproblem identifies the DNN split point, and the allocation of efficient communication resources is obtained by solving the second sub-problem. It also comprises a mixed global and local model parameter replication process that periodically backs up the model parameters to cope with scenarios such as worker failure or network disconnection to handle faults during DNN training. EcoFed (Wu et al., 2024a) is the first framework designed to effectively decrease communication overheads of DNN Partitioning-based FL (DPFL) using novel approaches for optimizing forward and backward passes in DPFL. EcoFed uses an activation switch to decide whether outputs from the device-side model need to be sent to the server or if the server can use the cached activation to train the server-side model from the replay buffer. Further, compression and quantization are used for the activation outputs when transmitting from the end device to ES. The solution updates the replay buffer with compressed and quantized data, which is reconstructed through the decompressor module. Thus, EcoFed minimizes the communication cost of DPFL by eradicating the gradient communication, adjusting activations' transmission frequency, and squeezing the activation data. Zhang et al. (2024b) designed a communication-efficient federated aggregation algorithm named FedSL, to diminish the communication overhead of FL-based DNN training while guaranteeing the performance of global model. The solution transmits fewer neural network parameters to reduce network data traffic. It determines the similarities of different layers of CNN models in FL to avoid redundant information in the feature extractor model, resulting in the transmission of lesser model parameters.

SL-based FL solution is designed for the Internet of Medical Things (IoMT) network by Ni et al. (2024) considering collaborative healthcare analytics framework to realize the training of the DL model in the eco-system of resource-deprived devices. The solution supports semi-distributed learning and preserves user data partially. It partitions the DNN model into two sub-networks, where high-level layers are executed on the server side, and lower layers are executed on IoMT devices to enable model training in deice-server synergy without revealing sensitive raw data. Wu et al. (2024b) discussed an SFL scheme for vehicular edge networks where the vehicular device's total cost was the weighted sum of the training time and energy consumption. The authors utilize a dynamic-step Q-learning-based RL solution to obtain the near-optimal allocation of the computation and communication resources and split the DNN between vehicular devices and ES. Arouj and Abdelmoniem (2024) designed EAFL+, a novel solution, to support computational intensive and delay-sensitive tasks in cloud and MEC-enabled environments. The approach integrates the SL paradigm with FL training to attain energy-aware training. SL-based computation offloading reduces low-energy devices' computational burden and enables them to participate in FL procedures in a heterogeneous computing environment. Hence, it enhances total training performance and lowers the dropout of clients. Luo and Zhang (2024) discussed an efficient distributed DNN training framework, which integrates the features of FL, SL, and mutual KD, respectively.

Discussion: The distributed edge training solution enable the collaboratively DNN training across multiple EDs, rather than depending on a centralized server. It leverages the computational power and data of multiple distributed EDs to accomplish DNN training task. Thus distributed edge training offers significant benefits in terms of privacy, scalability, fault tolerance, and resource utilization efficiency in compared to on-device edge learning. In the context of distributed DNN training enabler, FL-based solution obtains the global and generalized DNN model by aggregating the knowledge from local clients and SL-based solutions accomplish optimizes distributed DNN training.

FSL-based solutions leverage the features of SL and FL to train a generalized and optimized DNN model. The distributed training and privacy preserving features makes the FL as a potential model training solution across a wide range of applications. However, training a large DL model on resource-constrained node is still challenging. To this end, SL and FSL schemes realizes the distributed and privacy-preserving DL learning mechanism efficiently.

7.3. Data availability based DNN training in Edge-AI

The impact of data availability on DNN training tasks is overlooked in most of the Edge AI-based review articles. Thus, this section concerns the state-of-the-art methods of data availability based DNN training procedures, including Transfer Learning (TL) and Continual Learning (CL). Fig. 9 depicts the taxonomy of DNN training task under the limited data availability. TL enables the multi-task multi-model phenomena, which trains a new DNN for each new task with limited data. In contrast, CL enables multi-task single-model phenomena, which accommodates the knowledge of current task with limited data with the previously learned tasks in a single DNN. TL and CL solutions utilize the schemes depicted in Fig. 8 to accomplish DNN training task under limited data availability.

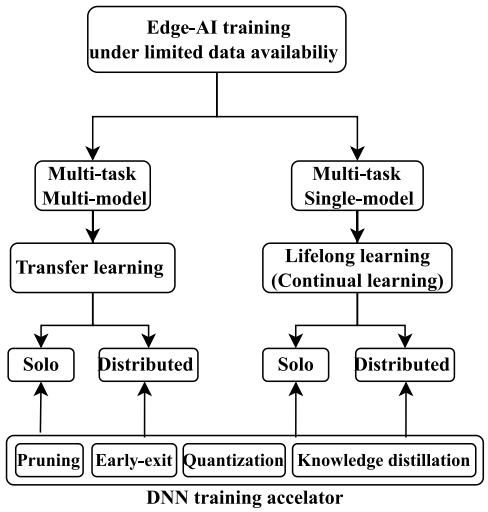


Fig. 9. Edge-AI training under limited data availability.

7.3.1. Transfer learning in Edge-AI

The DL techniques are data-dependent and thus mandate huge amounts of training data to train it from scratch. However, availability of limited labeled data is a major challenge in various applications such as smart health, manufacturing, etc. This issue of DL training with limited data can be preached by TL (Pan and Yang, 2009). TL has displayed remarkable capabilities in acquiring satisfactory DL performance through finetuning a DL model pre-trained for other tasks. Hou et al. (2020) deployed inspection robots and TL-based cascade mask region CNN on robots for identifying stay cable surface defects. Compared to traditional image processing and CNN, it offers a more accurate method for inspecting, precisely locating, and measuring surface defects. Wang et al. (2021) conducted a study detecting bearing faults in different working conditions having difficulty in obtaining large-scale labeled datasets. They employed the TL approach to transfer knowledge of fault diagnosis models trained under a single working condition to different working conditions with limited data samples. They combined a deformable CNN and deep LSTM for rolling bearing fault diagnosis. Xia et al. (2021) offered a DL solution for machine fault diagnosis, integrating Digital Twin (DT) and TL under limited labeled or inaccessible data for fault conditions of machines with differing working or shifted attributes. They validated it through a triplex pump fault diagnosis and developed a sparse denoising AutoEncoder (AE) that finetunes this model from the source domain to target working conditions. A TL-based edge-CNN framework was proposed by Yang et al. (2021a) for industrial edge networks to accomplish a balance between energy consumption, latency, and model performance. Yang et al. (2021b) integrated TL and an ensemble learning classifier to improve wind turbine blade damage recognition performance. This work employed TL to extract abstract features and fasten convergence efficiency. The random forest-based ensemble learning classifier improves the accuracy of detecting blade defects. Zim et al. (2023) developed a DNN model embedding TL for predicting the material removal rate of the electrical discharge machining process, which overcomes the requirement of large volumes of training data. Lu et al. (2022) developed an ensemble TL approach incorporating TrAdaBoost-R2, Bayesian, and Markov chain Monte Carlo calibration for cutting energy consumption prediction. Moreover, Lockner and Hopmann (2021) used induced network-based TL to reduce the huge training data requirement for an AI model trained for the injection molding process. They observed significant disparities in the transferability of the source model for varying geometries; likewise, similar geometries to the target process achieve the best performance parameters. TL can be validated using different DL models to detect defects in steel surfaces, wherein the MobileNet model for TL achieved the best detection rate of 80.41% and 96.94% for the SEVERSTAL and NEU datasets, respectively (Abu et al., 2021).

7.3.2. Continual learning in Edge-AI

In many applications, the training data used for the model and the new data often do not align. Therefore, a new model must be trained from scratch using new data or for a new task. Continual Learning (CL) is a recent learning paradigm that trains models over time to gain insight for new tasks and retain proficiency for previous tasks (Wang et al., 2024c). Thus, CL methods find a balance in the stability and plasticity of DL weights or parameters while training for new tasks under the unavailability of the previous task data. Recently, a few examples of CL have been encountered in EC and manufacturing separately, such as Tham and Sharma (2021) presented particle filter algorithm-based CL for the prescriptive maintenance method in a distributed factory environment. They used the partially observable MDP framework, and the overall system allows for the unique adaptation of each machine. Tercan et al. (2022) addressed the challenge of quality prediction of continuously changing production processes (manufacturing of new products) using the CL method named memory-aware synapses. The proposed mechanism successfully prevents the DL model from forgetting previous knowledge and improves the training performance for the new regression tasks in injection molding.

Nain et al. (2023) proposed a CL-enabled DNN model to be proficient in handling the changing predictive quality of multiple processes. To reduce the impact of task orders in sequentially arriving CL tasks, they introduced a lightweight similarity analysis module at the resource-limited edge. The proposed module can reduce task order's impact by 70% on the DNN performance after the fifth task in the use-case of plastic bricks. In their other work, the authors assumed different task importance under a mass production and customization scenario and developed a hybrid CL scheme named PackMASNet for the edge devices (Nain et al., 2024b). Maschler et al. (2022) discussed different regularization-based CL approaches on a real battery wear dataset, among which online elastic weight consolidation delivered the best performance. They further implemented CL approaches on NASA turbofan engine dataset and showed promising results for DL training (Maschler et al., 2020). Li et al. (2024b) developed a novel pseudo-replay CL method under the class incremental framework for anomaly detection. Training data scarcity of the current task plays a crucial role in achieving DL performance; therefore, Nain et al. (2026) designed a frugal AI solution that uses data augmentation

Table 4
Summary of Edge-AI empowered DNN training solutions.

Reference	Architecture	DNN type	Training accelerator	Training delay	Training energy	Device synergy	Memory footprint	Data privacy	Performance evaluation
Tang et al. (2022)	Solo	CNN	Early-exit	Yes	No	None	Yes	Yes	Obtains accuracy gain up to 32.7%
SK et al. (2022)	Solo	CNN	None	Yes	Yes	None	–	Yes	–
Wang et al. (2023b)	Solo	BNN	None	Yes	No	None	Yes	Yes	Memory requirement reductions of 3–5×, while reaching similar test accuracy
Kook et al. (2024)	Solo	DNN	None	Yes	Yes	None	–	Yes	–
Yu et al. (2024)	Solo	CNN	Pruning	Yes	No	None	Yes	Yes	Attains a model size reduction of 60%
Sung and Han (2024)	Solo	CNN	Pruning	Yes	No	None	Yes	Yes	–
Hamad (2025)	Solo	ML	None	No	No	None	Yes	Yes	–
Zeng et al. (2020)	Collaborative	DNN	FL	Yes	Yes	UAVs	No	Yes	Reduce the number of communication rounds needed for convergence by 35%
Brik et al. (2022)	Collaborative	RNN	FL	Yes	Yes	FL clients	No	Yes	–
Li et al. (2022b)	Collaborative	CNN,RNN	FL	Yes	No	Device-ED	No	Yes	–
Farahani and Monsefi (2023)	Collaborative	CNN,RNN	FL	No	No	FL clients	No	Yes	–
Putra et al. (2023)	Collaborative	CNN	FL	Yes	No	Fog-Cloud	No	Yes	Attains 24% faster training time
Tonellotto et al. (2021)	Collaborative	LSTM	FL, Quantization	Yes	No	EDs-Cloud	Yes	Yes	Obtains up to 19 times reduction of volume of data exchanged
Zheng et al. (2021)	Collaborative	CNN	FL	Yes	Yes	ED-Cloud	No	Yes	–
Wang et al. (2020d)	Collaborative	CNN	FL,KD	Yes	No	Fog-Cloud	Yes	Yes	–
Zhang et al. (2020)	Collaborative	DNN	FL, Early-exit	Yes	No	IoT-Cloud	Yes	Yes	–
Tanghatari et al. (2022)	Collaborative	CNN	FL, Quantization	Yes	No	IoT-ES	Yes	Yes	Preserve data privacy, on average, about 3.5% accuracy loss
Mamba Kabala et al. (2023)	Collaborative	CNN	FL	Yes	No	–	–	Yes	–
De Rango et al. (2023)	Collaborative	CNN	FL	Yes	Yes	Device-EDs	Yes	Yes	–
Pervej et al. (2023)	Collaborative	–	FL	Yes	Yes	Device-ES	–	Yes	–
Lian et al. (2023)	Collaborative	CNN	FL, Sparsification	Yes	No	IoT-ED	Yes	Yes	–
Djenouri et al. (2023)	Collaborative	GCNN	FL	Yes	No	–	–	Yes	Model attains average precision of 84%
Aggarwal et al. (2024)	Collaborative	CNN	FL+TL	Yes	No	IoAT devices	Yes	Yes	–
Mwawado et al. (2024)	Collaborative	CNN	FL	Yes	No	EDs-Server	–	Yes	–
Bera et al. (2024)	Collaborative	DNN, RNN	FL	Yes	Yes	IoT-Dew-ED-Cloud	Yes	Yes	Achieves ≈99% prediction accuracy at ≈50% lower latency and energy consumption
Zhang et al. (2024a)	Collaborative	DNN	FL, Quantization	Yes	No	FL clients	Yes	Yes	–
Adjei-Mensah et al. (2024)	Collaborative	CNN	FL, Attention	Yes	No	–	–	Yes	Obtains 86.7% average accuracy
Gupta and Raskar (2018)	Collaborative	CNN	SL	No	No	Device	–	Yes	–
Zhang and Xu (2021)	Collaborative	CNN	SL	Yes	No	Device-ES	–	Yes	–
Bharti and McGibney (2022)	Collaborative	ANN	SL	–	Yes	Device-ES	–	Yes	–
Akintoye et al. (2022)	Collaborative	CNN	SL	Yes	No	EDs	–	Yes	Performs 20% averagely better in terms of training time
Samikwa et al. (2022)	Collaborative	CNN	SL	Yes	Yes	Device-ES	–	Yes	Accelerates model training on IoT devices by up to 48% and minimizes the energy consumption by up to 61.4%
Wang et al. (2023d)	Collaborative	CNN	SL	Yes	No	EDs	Yes	Yes	–
Li et al. (2023b)	Collaborative	CNN	SL	Yes	No	EDs	Yes	Yes	Saves 57% of memory overhead and achieving 4.18× memory-efficiency
Lien et al. (2024)	Collaborative	CNN	SL	Yes	Yes	Device-ED	–	Yes	–

(*continued on next page*)

within CL. Fault detection applications in industrial scenarios have some unexpected faults that appear in the actual working conditions, called incremental fault categories or classes. Training traditional DL models requires collecting all possible data for retraining, since using only new fault data leads to catastrophic forgetting of old tasks. Liu et al. (2023a) addressed this through a lifelong learning method using generative models in replay-based CL schemes for bearing fault diagnosis. The generator produces old task features mixed with features of recent or new tasks to overcome the imbalance and forgetting-related issues in the classification. A similar problem has been resolved by

Table 4 (*continued*).

Huang et al. (2024)	Collaborative	CNN	SL	Yes	No	Devices	–	Yes	–
Al Maruf et al. (2024)	Collaborative	CNN	SL	Yes	No	EDs	–	Yes	Reduces overall model training time by ≈44.87%
Guo et al. (2021)	Collaborative	CNN	SFL	Yes	No	EDs	–	Yes	–
Bharti and Mcgibney (2021)	Collaborative	ANN	SFL	–	–	EDs	Yes	Yes	–
Liu et al. (2022)	Collaborative	CNN	SFL	Yes	No	Device-ED	–	Yes	–
Deng et al. (2022)	Collaborative	CNN	SFL	Yes	No	Device-ED	–	Yes	–
Cheng et al. (2023)	Collaborative	CNN	SFL	Yes	No	Device-ES	–	Yes	–
Feng et al. (2023)	Collaborative	CNN	SFL	Yes	Yes	ED-Cloud	–	Yes	–
Xu et al. (2023a)	Collaborative	CNN	SFL	Yes	No	Device-server	–	Yes	–
Khan et al. (2023)	Collaborative	DNN	SFL	Yes	No	Device-EDs	–	Yes	–
Chen et al. (2023c)	Collaborative	CNN	SFL	Yes	No	Device-ED	Yes	Yes	Achieves 6.8x faster training time
Guo et al. (2023)	Collaborative	CNN	SFL	Yes	No	Device-ED	–	Yes	–
Wu et al. (2024a)	Collaborative	CNN	SFL, compression	Yes	No	EDs	–	Yes	Reduces communication cost from 21 times to 133 times
Zhang et al. (2024b)	Collaborative	CNN	SFL	Yes	No	–	–	Yes	Decreases 42% communication cost
Ni et al. (2024)	Collaborative	CNN	SFL	Yes	No	Devices	–	Yes	–
Wu et al. (2024b)	Collaborative	CNN	SFL	Yes	Yes	Devices	–	Yes	–
Arouj and Abdelmoniem (2024)	Collaborative	DNN	SFL	–	Yes	Device-ED-Cloud	–	Yes	Enhances accuracy up to 9–24%
Luo and Zhang (2024)	Collaborative	CNN	SFL,KD	Yes	No	–	–	Yes	–
Hou et al. (2020)	Solo	RCNN	TL	No	No	None	No	Yes	–
Xia et al. (2021)	Solo	AE	TL	No	No	None	No	Yes	–
Yang et al. (2021b)	Solo	CNN,ML	TL	No	No	None	No	Yes	–
Lu et al. (2022)	Solo	ML	TL	No	Yes	None	No	No	–
Lockner and Hopmann (2021)	Solo	ANN	TL	No	No	None	No	No	–
Abu et al. (2021)	Solo	CNN	TL	No	No	None	No	No	Obtains high detection rate of 80.41% and 96.94%
Tham and Sharma (2021)	Solo	ML	CL	No	No	None	No	Yes	Achieve speed-ups of 4 to 20 times
Tercan et al. (2022)	Solo	ANN	CL	No	No	None	No	No	–
Nain et al. (2023)	Solo	ANN	CL	No	No	None	No	Yes	Validate the impact of task order and reduce this impact by 70% on the model's overall performance
Nain et al. (2024b)	Solo	ANN	CL	Yes	No	None	No	Yes	Saves computation cost by 132 times, reduces the DNN training time by 20.45%
Maschler et al. (2022)	Solo	RNN	CL	No	No	None	No	No	–
Maschler et al. (2020)	Solo	AE,RNN	CL	No	No	None	No	No	–
Li et al. (2024b)	Solo	DNN	CL	No	No	None	No	No	–
Liu et al. (2023a)	Solo	CNN	CL	No	No	None	No	No	–
Chen et al. (2022b)	Solo	DNN	CL						–
Zizzo et al. (2022)	Collaborative	DNN	CL,FL	No	No	FL clients	No	Yes	Achieves maximum performance gains of 49%
Zhang et al. (2023b)	Collaborative	CNN	CL,FL	Yes	No	FL clients	No	Yes	–
Zhang et al. (2023a)	Collaborative	CNN	CL,FL	No	No	FL clients	No	Yes	–
Wang et al. (2024a)	Collaborative	CNN	CL,FL	Yes	No	FL clients	Yes	Yes	–
Gauttam et al. (2024)	Collaborative	CNN	CL,SL	Yes	No	EDs	No	Yes	Reduces model maintenance cost from 12 to 48 percentage

Chen et al. (2022b) using a Dual-Branch Aggregation Networks (DBANets) framework that combines the reserved examples of previous tasks or faults while learning new faults to avoid forgetting the old faults. An improved replay-based CL scheme is discussed to cater to the various skews, such as uneven class distribution, varying client participation, etc., under FL setting (Zizzo et al., 2022). Zhang et al. (2023b) integrate the CL into FL (termed as FCL) to address the performance degradation on non-independent identically distributed data across clients under heterogeneous data distributions. Further, a bidirectional compression and error compensation scheme is discussed to minimize the communication overhead. TARGET, as devised by Zhang et al. (2023a), is a novel federated class incremental CL that dynamically accommodates the new classes in FL. The global model utilizes the KD to transfer the knowledge of previous tasks to the current task at the model level. Wang et al. (2024a) presented a traceable FCL scheme to tackle model performance degradation due to task repeatability among dynamic data owing clients in FL. Gauttam et al. (2024) discussed SCL, a hybrid solution that utilizes the characteristics of SL and CL to attain a sustainable DL solution in near-real-time.

Discussion: Traditionally, DL models are trained using historical data under the assumption that model performance remains the same over time. However, the potential application use cases in smart manufacturing, connected health, smart city, precision agriculture, etc., exhibit the data changing trends over time, which initiates the need for model retraining. TL-based solutions address the aspect of DNN training under limited data availability for multi-task multi-model scenarios. CL-based solution enables the multi-task single-model phenomena to ensure dynamism and adaptability metrics, where a single model learns the knowledge of new tasks using current data without forgetting the learning of previous tasks. TL and CL-based schemes emerged as the key enabler for DL model training under limited data availability and varying data scenarios for applications such as smart manufacturing, smart city, etc. Table 4 represents the summary of the contemporary edge training schemes.

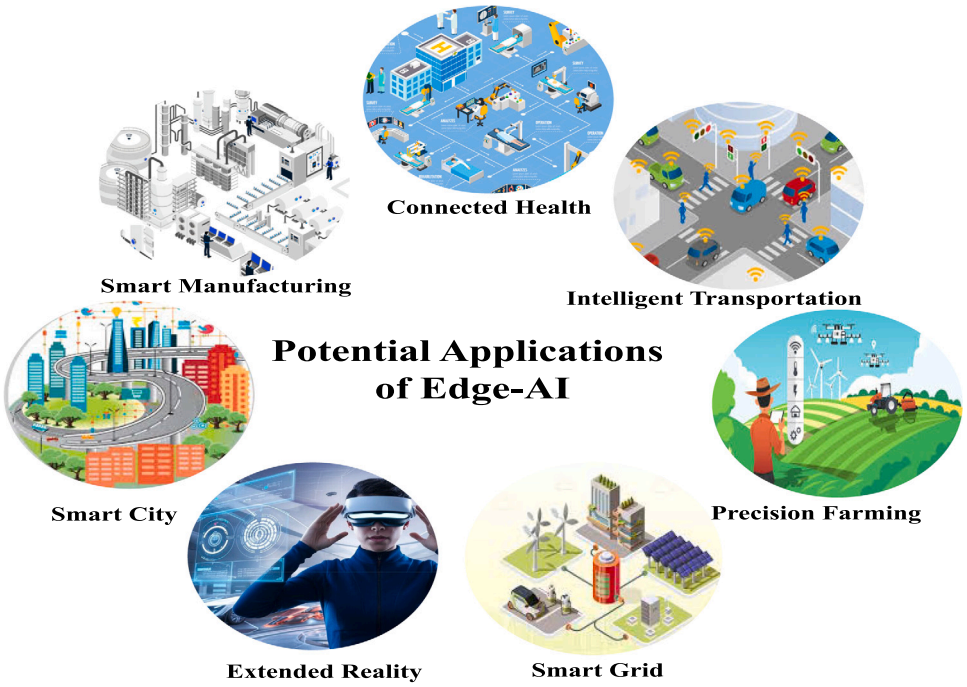


Fig. 10. Edge-AI application.

8. Potential applications domains

The applicability scope of intelligent solutions (such as DL based solutions) comprises a wide set of application domains including connected health, intelligent transportation, smart manufacturing, smart city, precision farming etc. The performance metrics of various use-case solutions within these applications are characterized as real-time response, data-privacy, energy consumption, adaptability, dynamism, multi-modal data fusion, scalability, and reliability, respectively. The use-case specific applicability of Edge-AI solutions in various application domains is discussed below and illustrated in Fig. 10.

Smart Manufacturing: The smart manufacturing solutions integrates the intelligence (such as DL models) with traditional manufacturing processes to increase system productivity, efficiency, and revenue. The potential Edge-AI use-cases for the smart manufacturing comprises predictive maintenance, quality inspection, autonomous operations, environmental monitoring, and mixed reality applications etc. The industrial data comprises the important information such equipment health status, product information, etc. The Edge-AI solutions utilizes this information at edge for decision making, which avoid its exposure to the third party and preserve the data-privacy. Real/near-real time response is a key requirement for the above mentioned use-case scenarios to enhance productivity and efficiency. For instance, predictive maintenance solutions is a vital component in smart manufacturing, which predict equipment failures before they occur, on the basis of various operational parameters of respective equipment. The Edge-AI empowered predictive maintenance solution attains quick and near-real-time decision than cloud-based solution, which provides a longer time span for maintenance planning and reduces the risk to human life. Similarly, inference acceleration based Edge-AI solutions increases the throughput of quality inspection system. The smart factory floor comprises a set of equipment, which continuously produces operational health parameters based on their operating conditions. The Edge-AI empowered distributed learning paradigm (such as FSL) enable the extraction of various key health patterns at equipment level (such as local model training in FL) and aggregate them into a global model, resulting a more generalized predictive maintenance solution in a near-real-time. The analogy is also applicable to other use-cases. Furthermore the industrial data in susceptible to changes (such as change in data distribution, arrival of new product types, etc.), which may hamper the application performance in the discussed use-case scenarios. The Edge-AI empowered LLL solutions updates the DL model by adapting the data dynamism in a near-real-time to design a robust and sustainable smart manufacturing solution.

Connected Health: The connected health data encompasses the vital and sensitive information such as patient health status, test reports, prescriptions, patient personal information, etc. Edge-AI solutions process the sensitive information locally to ensure data-privacy. The various DL-based use-cases of connected health including remote patient monitoring, robotic surgical assistance, emergency response, etc., demands near-real time responsiveness and high precision model accuracy for robust decision making, which can be attained using Edge-AI solutions (refer to section edge inference). The distributed Edge-AI learning empowers the generalized learning by aggregating the knowledge of various local client in scenario such as spread of a particular flu, early detection of rare disease, etc. Moreover, the integration of LLL paradigm with Edge-AI training solutions enable the robust and sustainable connected health solution in dynamic data scenario such change in the mapping of new treatment and prescription to the disease.

Intelligent Transportation: Autonomous vehicles are the future of vehicular technology. The potential application use-case comprises autonomous driving, traffic management, congestion control, smart parking, predictive maintenance of vehicles and infrastructure, etc., and utilizes AI-based solutions for decision making. The near-real-time responsiveness is an essential requirement to ensure robust operational efficiency in the majority of use cases, which is addressed by the Edge-AI solutions, such as inference acceleration schemes at the edge. The data processing at the edge mitigates the privacy violation risks of sensitive information such as user personal information, driving patterns, behavioral patterns, etc. The distributed learning in the Edge-AI paradigm empowers generalized global knowledge learning by aggregating the local information of various vehicles in various regions to attain improved traffic management, reduced congestion, search time minimization in smart parking, etc., without data privacy violation. Moreover, the Edge-AI and LLL-based hybrid solutions enable

the intelligent transportation systems to handle dynamic environments, new data, and emerging challenges (such as new traffic patterns, road conditions, weather conditions, etc.).

Smart City: The smart city solutions aim to design a sustainable and optimized urban infrastructure for the residents. The key applications include intelligent traffic management, smart and sustainable buildings, public safety and security, waste management, smart water and energy management, etc. The Edge-AI solutions extract the key patterns from various sensors (temperature, pressure, camera, etc.) at the network edge to enable near-real-time decision making and mitigate data privacy violations. The distributed Edge-AI learning solutions enable efficient, scalable, and generalized learning in smart city use-case applications. For instance, in intelligent traffic management, the local model learns the knowledge of traffic patterns within a region. The knowledge of various regions' local models is aggregated at a centralized global model to attain efficient intelligent traffic management across the city. Furthermore, the data of various smart city applications is susceptible to continuous changes, such as traffic patterns, environmental changes, people's behavior, etc., which hamper system performance. The CL empowered Edge-AI learning solutions can be utilized to cater to the same.

Precision Farming: The precision farming solutions are revolutionizing the traditional agricultural process through data-driven decision making for increased efficiency and productivity. The potential use case involves crop health monitoring, smart irrigation, intelligent pest control, predictive health maintenance of agricultural machinery, etc. Edge-AI solutions process sensitive data, such as crop health and livestock monitoring, at the edge to preserve data privacy and utilize solutions such as inference acceleration schemes to attain decision-making in near-real time. Furthermore, various agricultural processes exhibit varying data patterns such as crop growth patterns, soil health, crop disease growth, etc. The distributed Edge-AI learning solutions obtain a generalized global model by accommodating the knowledge of various local models for efficacious decision-making. Furthermore, the productivity of agricultural solutions is directly impacted by the environmental parameter, weather conditions, water level data, etc., which reveals dynamic patterns. Thus, Edge-AI empowered CL solutions can be utilized to attain sustainable and productive manufacturing solutions.

Smart Grid: A smart grid is an advanced electricity network that integrates digital communication, automation, and real-time data analytics to enhance energy distribution and consumption efficiency, reliability, and sustainability. Edge AI in smart grids enables real-time decision-making, low latency, reduced bandwidth usage, greater reliability (in case of network failures), enhanced efficiency, and improved privacy by processing data closer to the source. These capabilities make Edge AI a key enabler for the next generation of intelligent and resilient smart grids with key applications such as predictive maintenance, demand response optimization, anomaly detection, etc. Edge AI analyzes sensor data from transformers and power lines to detect faults before they cause failures, reducing downtime and maintenance costs. The demand response can be optimized in a smart grid using AI at the edge to predict energy consumption patterns and dynamically adjust power distribution, enhancing grid stability. Anomaly detection is also crucial, as Edge AI can identify cybersecurity threats or power theft in real-time, improving grid security. Additionally, renewable energy integration benefits Edge AI by balancing power fluctuations from solar or wind sources locally, ensuring efficient energy utilization. The major advantages of Edge AI in these scenarios include low latency, as decisions are made instantly without relying on cloud processing; enhanced privacy, since sensitive energy data is processed locally; reduced bandwidth usage, minimizing the need for large-scale data transmission; and greater reliability, as edge devices can continue functioning even in the case of network failures.

Extended Reality (XR): It encompasses Virtual, Augmented, and Mixed Reality, when associated with Edge AI and accelerating hardware, marks its future. Its key application is real-time object recognition and interaction, where Edge AI instantly identifies objects, gestures, or faces, making XR-based industrial training, healthcare, and retail more interactive. Edge AI enhances realism in immersive gaming and virtual environments by dynamically adjusting rendering based on player movements. AI-driven non-playable characters can respond in real time, leading to smoother gameplay and energy-efficient processing. Smart navigation and assisted reality use AR-powered navigation apps to guide users in smart cities, airports, and industrial facilities, where Edge AI ensures reliable offline operation, instant updates, and context-aware personalization. For remote collaboration, such as AR-based remote assistance in telemedicine or industrial repair, Edge AI enables real-time object tracking, gesture recognition, and instant language translation, making communication seamless. Additionally, adaptive content streaming and rendering powered by Edge AI optimizes XR content delivery based on network conditions and device capabilities, ensuring smooth performance while minimizing bandwidth usage.

Edge-AI solutions in the above-discussed domains offer benefits such as reduced latency, improved reliability, enhanced privacy, lower bandwidth requirements, etc., making it a game-changer in diverse use-case scenarios as discussed in Table 5.

9. Limitations and future directions

The section discusses the key limitations and potential future directions for designing a robust and sustainable Edge-AI framework.

Cost-effective and scalable computing infrastructure: The Majority of existing ED placement strategies work with the assumption of a fixed number of EDs and address the issues of latency minimization, load balancing, and reduction of energy consumption. However, the scalable deployment of EDs needs to be addressed in view of the rapid proliferation of IoT devices to attain sustainable performance. Moreover, the number of EDs directly affects the deployment cost. Hence, inclusion of the number of EDs as a decision parameter in the ES placement scheme has a significant impact on the reduction of computing infrastructure deployment cost. Furthermore, the occurrence of faults at EDs degrades the reliability of DL task execution. It can be addressed by developing fault-tolerant edge infrastructure deployment schemes.

Varying data traffic, mobility, and intermittent network connectivity: The device mobility affects the IoT-ED mapping, which significantly affects latency and resource utilization. The variation of data traffic (due to event/query-driven scenarios and device mobility) influences the data traffic distribution among EDs. Moreover, the intermittent network connectivity may impose additional computational overhead to recompute the information lost during network disconnection. Thus, the future solutions need to address these issues to design an efficient edge computing infrastructure. In distributed training and inference schemes, such as FL and SCL, latency and model performance variance are observed due to heterogeneous device capabilities, mobility, varying network connectivity, and asynchronous participation of edge devices. It impacts DL solutions' convergence and responsiveness, which latency-aware aggregation strategies and adaptive scheduling can manage.

Topology maintenance in computing infrastructure: It handles the association of devices to the respective EDs. The inclusion of new devices and exclusion of existing devices (such as due to energy drain) imposes the challenge of device-ED remapping to meets the QoS requirements. Furthermore, the occurrence of faults at EDs can be addressed by assigning the nodes among other EDs, which also triggers the topology maintenance. The issues need to be addressed to ensure

Table 5
Comparative summary of datasets, models, and performance across different application domains.

Application	Work	DEMMA layer	Dataset	Models used	Performance metrics	Performance evaluation
Connected health	Gupta and Raskar (2018)	Layer 3	MNIST, CIFAR10, ImageNet	CNN	Accuracy, data privacy	Achieved accuracy of 57%–99% in a privacy preserving fashion through distributed learning of deep neural network over multiple agents
	Xu et al. (2019)	Layer 3	MNIST, Word vectors	CNN, RNN	DNN inference delay	Attains 5.08 times and 23.0 times execution speedup, as well as 53.5% and 85.5% energy saving compared to wearable-only and handheld-only strategies
Intelligent transportation	Liu et al. (2023b)	Layer 3	ImageNet	CNN	DNN Inference delay and reliability	Achieves 1/2-optimal approximation on maximizing the inference reliability
	Li et al. (2023a)	Layer 3	CeyMo road marking dataset	CNN	DNN Inference delay	The delay obtained by the proposed scheme is 81.42%, 70.68%, and 55.59% lower than local, IMOPSOQ, and EHGA algorithms, respectively.
Smart manufacturing	Gauttam et al. (2022c)	Layer 2	Network data traffic	–	Latency, data traffic distribution	Reduces IoT nodes delay by 23%–64%, 20%–66%, and 35%–73% on periodic, event-based, and query-based data traffic scenarios, respectively, under various network settings, compared to the benchmark solutions.
	Nain et al. (2024b)	Layer 3	Plastic bricks dataset	ANN	Backward, forward, and overall Loss	Reduces the DNN training time by 20.45%, and improves overall performance by 43.60% over the SOTA schemes under limited data availability.
	Gauttam et al. (2024)	Layer 3	Nut dataset	CNN	Model maintenance cost (accuracy and model training time)	Attains a model maintenance cost reduction of 12%–48% and 18%–36% under static and mobile scenarios with limited data availability compared to SOTA schemes.
Precision farming	Rakib et al. (2024)	Layer 3	PlantVillage dataset	CNN	Accuracy, Memory footprint	Achieved 98% accuracy with 28 kB model size.
	Jiao et al. (2024)	Layer 3	Litchi dataset	CNN	Accuracy, Precision, recall, Memory footprint	Achieves the model size of is 6.9 MB with 94.9% average precision and 97.2% average recall
Smart city	Jiang et al. (2023)	Layer 2	Network data traffic	DRL	Edge server cost	Reduces edge server cost by 13.20% to 61.84% and 23.09% to 66.32%, respectively
	Khan et al. (2023)	Layer 3	MNIST and Fashion MNIST data sets	CNN	Latency, energy, data privacy	Reduces edge server computing delay, transmission delay, and energy consumption compared to SOTA
	Samikwa et al. (2023)	Layer 3	Imagenet	CNN	DNN inference delay, energy consumption	Reduces DNN inference delay and energy consumption by up to 5.2× and 6×, respectively, compared to SOTA without loss of accuracy

non-disruptive service execution under the scenarios of inclusion and exclusion of devices and faulty EDs.

Model maintenance (update) at edge: Model maintenance is a critical task in the Edge-AI ecosystem, which ensures the reliability and performance of the given DL model over time. The current state-of-the-art Edge-AI solution operates under the traditional DL assumption that DNN trained on historic data set model will remain efficacious and functional indefinitely. However, the dynamic nature of data due to alterations in data distribution, fluctuations in the input–output mapping relationship over time may degrade the model performance, which initiates the model update/maintenance. Traditionally, the model maintenance mechanism comprises the model retraining from scratch using the current and historical data. The procedure imposes the computational burden on resource-constrained EDs. To this end, FL, a distributed learning approach, empowers the model learning (update) in a distributed fashion. However, execution of model update using FL in a heterogeneous ED suffers from issues such as model staleness, stragglers, etc., due to the varying resource (computing and storage)

capabilities of EDs. In addition, EDs' mobility and varying network connectivity among the FL clients also affect the model update cost at the edge. The above-discussed concerns require immediate attention towards the design and development of edge empowered model update schemes. Furthermore, the availability of limited data at the edge due to governance regulations, policies, and storage limitations imposes additional challenges to model updates.

TL and CL integration in AI model training/update: TL-based solution enables multi-model multi-task learning with limited data availability, whereas CL-based solution empowers the single-model multi-task learning under the availability of current-task data. However, the scheme comprises an approximately similar computational burden as regular DNN training. Thus, integration of TL and CL-based scheme with existing DNN training acceleration methods leads to the development of sustainable Edge-AI solutions in near-real-time. Tradeoffs between the storage, computing time, and DL solution performance on the tasks must be determined in these learning mechanisms for specific application use-cases.

Reliability: The Edge-AI paradigm encompasses the various components, such as EDs, communication links among EDs, and services (such as DNN models) hosted at EDs. For instance, an Edge-AI empowered machine vision system consists of an end node (camera), ECN, ES, a DNN model, and a communication link between ECN and ES, respectively (Gauttam et al., 2022a). The malfunctioning of any of these components may degrade the reliability performance of the system. Thus, the reliability aspects need to be addressed at both infrastructure (edge computing) and application (DNN models) levels to accomplish the objective of a robust and sustainable Edge-AI framework. This becomes even more critical in collaborative training and inference in federated learning settings.

Inclusion of various modality DNNs: The majority of the existing works implements the edge inference and training acceleration schemes on the sequential DNN model (including CNN and ANN), which questions the acceptability of the schemes for generic DNNs. Thus, the efforts need to be made in the direction of implementing the scheme with other DNNs, including GNN, time series DNN, auto-encoders, etc., to extend the scope of schemes for various modality DNNs and ensure the generality. Complex and advanced models, such as transformers and generative AI models used in different applications, must be validated for applicability within Edge-AI orientation. This can enhance the vision and understanding of the comparative improvement in terms of storage, computations, memory, inference, and training time for different DL modalities.

Multi-objective solutions: Each of the components of the Edge-AI ecosystem comprises multiple performance indicators. For instance, the performance of edge infrastructure deployment solutions is measured in terms of delay, load balancing, resource utilization, etc. Similarly, DNN tasks (both training and inference) performance metrics include inference/training latency, energy consumption, model accuracy, memory footprint, etc. Majority of existing edge infrastructure and DNN task execution solutions focus on the optimization of a single performance indicator. Thus, there is a need to design the multi-objective solutions for optimized edge infrastructure deployment and efficient DNN task execution to attain the trade-off among performance indicators.

Data privacy and security aspects: The Edge-AI solutions in various application scenarios handle sensitive data, such as health records, location information, driving behavior, product quality information, etc. It initiates the need to design an efficient solution to preserve data privacy while sharing the data among collaborative devices. Furthermore, the distributed nature of Edge-AI solutions and diverse jurisdiction imposes additional challenges in the implementation of uniform privacy policies. The additional Edge-AI security challenges include vulnerable EDs, AI model inversion, data poisoning attacks, model theft, etc. Thus, designing efficient data privacy and security solutions will be a promising future direction to attain a robust Edge-AI paradigm.

Human in the loop: Integration of human-in-the-loop concept in Edge AI aimed to leverage both human intelligence and AI at the network's edge. The human in the loop plays a viable role under contextual decision-making, real-time feedback, ethical considerations, etc.. Moreover, in the context of DNNs, the assistance of humans in the loop concept may be incorporated for tasks such as data labeling to train a model, validating model performance, etc. The inclusion of humans in the loop ensures efficiency and reliability in the above-mentioned potential application domains.

Edge-AI as a service: The Edge-AI solutions exhibit promising and efficient results for various application use-cases in the research phase. However, accommodating the Edge-AI research solutions into practice comprises many challenges. For instance, efficient deployment of an Edge-AI solution necessitates seamless collaboration among AI service providers, edge infrastructure vendors, network operators, data providers, and end-users. It initiates the need for a global model to enable the interaction among various entities. The other potential research direction includes the data ownership, sharing, and protection standardization among collaborative entities, standardized deployment frameworks design, development of scalable model deployment solutions, etc.

10. Conclusion

This survey explored and encompassed the multifaceted and emerging domain of Edge-AI, encompassing critical aspects such as edge infrastructure planning, DL inference, and training methodologies in a resource-constrained edge environment. The synergy among these aspects highlights the evolution of intelligent systems capable of performing complex DL tasks at the network's edge, minimizing latency, reducing bandwidth usage, and enhancing data privacy. The work proposed a layered Edge-AI framework, DEMMA, to conduct a systematic and detailed component-level investigation of various aspects required to be addressed to design an efficient Edge-AI framework. It also discussed a mathematical analysis to validate the adoption of layered architecture to represent an Edge-AI framework. The discussion in the proposed survey is modeled around optimized edge infrastructure and efficient DL task execution strategies (represented by layer 2 and 3 in Fig. 3). The work presented a detailed discussion on optimized edge infrastructure deployment strategies, encompassing EDs placement, node-ED mapping, load balancing among EDs, etc. The Technology Maturity Model (TMM) for edge infrastructure deployment is expected to evolve from static, heuristic-based deployment towards fully autonomous, AI-driven optimization, over the next 3–5 years. The edge infrastructure solution will utilize the emerging techniques such as containerization, orchestration, AI schemes (such as FL, reinforcement learning), fault-aware EDs placement, etc., to create a sustainable, self-optimizing, self-healing infrastructure. Further, it introduced an offloading-based categorized discussion of DNN task execution solutions. In the context of DL inference tasks, the analysis of schemes executing inference tasks locally (at the end or edge node) using DNN accelerators (such as pruning, quantization, etc.) is presented under binary offloading. In contrast, distributed collaborative solutions for DNN inference tasks (such as model split) are explored under partial offloading. The Edge-AI inference solution is widely used across various application use-cases, including smart manufacturing, healthcare, intelligent traffic systems, etc., operating at TMM level 2. Over time, it is expected to make a shift towards fully autonomous, adaptive, context-aware, and trustworthy inference ecosystems from current isolated, hardware-dependent deployments. The work utilized the same offloading-based analogy, comprising solo (on-device training) and distributed (FL, SL, etc.) DL training solution to describe the existing DL model training optimization solutions in the Edge-AI paradigm. It

also explored the optimized DL model training solutions under limited data availability. The current Edge-AI training scheme, such as FL, is utilized across various pilot projects for various application use-cases. The evolution of Edge-AI training over the next 3–5 years spans in the context of TMM includes the development of standardized federated learning pipelines, collaborative model updates, self-adapt to local data distributions using TL/CL, a trustworthy, explainable, and privacy-preserving ecosystem. Finally, the key limitations and potential future direction are discussed to design a robust, sustainable, and efficient Edge-AI paradigm. By advancing these areas, Edge-AI stands poised to revolutionize various application use-cases, enabling intelligent, decentralized systems that meet the demands of an increasingly connected world.

CRediT authorship contribution statement

Himanshu Gauttam: Writing – review & editing, Writing – original draft, Visualization, Methodology, Funding acquisition, Conceptualization. **Garima Nain:** Writing – review & editing, Writing – original draft, Conceptualization. **K.K. Pattanaik:** Writing – review & editing, Writing – original draft, Supervision. **Paulo Mendes:** Writing – review & editing, Writing – original draft, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the Indo-German Science and Technology Centre (IGSTC) Industrial Fellowship program [Grant number: IGSTC/IF Call 2024/PDIF/HG-3020/21/2024-25].

Data availability

No data was used for the research described in the article.

References

Abu, M., Amir, A., Lean, Y., Zahri, N., Azemi, S., 2021. The performance analysis of transfer learning for steel defect detection by using deep learning. In: Journal of Physics: Conference Series. vol. 1755, IOP Publishing, 012041.

Adjei-Mensah, I., Zhang, X., Agyemang, I.O., Yussif, S.B., Baffour, A.A., Cobbinah, B.M., Sey, C., Fiasam, L.D., Chikwendu, I.A., Arhin, J.R., 2024. Cov-Fed: Federated learning-based framework for COVID-19 diagnosis using chest X-ray scans. Eng. Appl. Artif. Intell. 128, 107448.

Aggarwal, M., Khullar, V., Goyal, N., Prola, T.A., 2024. Resource-efficient federated learning over IoAT for rice leaf disease classification. Comput. Electron. Agric. 221, 109001.

Ahmad, S., Shakeel, I., Mehfuz, S., Ahmad, J., 2023. Deep learning models for cloud, edge, fog, and IoT computing paradigms: Survey, recent advances, and future directions. Comput. Sci. Rev. 49, 100568.

AI and machine learning products, 2025. https://cloud.google.com/products/ai.

Akintoye, S.B., Han, L., Zhang, X., Chen, H., Zhang, D., 2022. A hybrid parallelization approach for distributed and scalable deep learning. IEEE Access 10, 77950–77961.

Al-Dulaimy, A., Jansen, M., Johansson, B., Trivedi, A., Iosup, A., Ashjaei, M., Galletta, A., Kimovski, D., Prodan, R., Tserpes, K., et al., 2024. The computing continuum: From IoT to the cloud. Internet Things 27, 101272.

Al Maruf, M., Azim, A., Auluck, N., Sahi, M., 2024. Optimizing DNN training with pipeline model parallelism for enhanced performance in embedded systems. J. Parallel Distrib. Comput. 190, 104890.

Alzubaidi, L., Zhang, J., Humaidi, A.J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M.A., Al-Amidie, M., Farhan, L., 2021. Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions. J. Big Data 8, 1–74.

Arouj, A., Abdelmoniem, A.M., 2024. Towards energy-aware federated learning via collaborative computing approach. Comput. Commun. 221, 131–141.

Asghari, A., Azgomi, H., Zoraghchian, A.A., Barzegarinezhad, A., 2024. Energy-aware server placement in mobile edge computing using trees social relations optimization algorithm. J. Supercomput. 80 (5), 6382–6410.

Asghari, A., Sayadi, M., Azgomi, H., 2023a. Energy-aware edge server placement using the improved butterfly optimization algorithm. J. Supercomput. 79 (13), 14954–14980.

Asghari, A., Vahdani, A., Azgomi, H., Forestiero, A., 2023b. Dynamic edge server placement in mobile edge computing using modified red deer optimization algorithm and Markov game theory. J. Ambient Intell. Humaniz. Comput. 14 (9), 12297–12315.

Ashiquzzaman, A., Oh, S., Lee, D., Lee, J., Kim, J., 2020. Compact deeplearning convolutional neural network based hand gesture classifier application for smart mobile edge computing. In: 2020 International Conference on Artificial Intelligence in Information and Communication. ICAIIC, IEEE, pp. 119–123.

Azure AI solutions, 2025. https://azure.microsoft.com/en-us/solutions/ai.

Baccour, E., Mhaisen, N., Abdellatif, A.A., Erbad, A., Mohamed, A., Hamdi, M., Guizani, M., 2022. Pervasive AI for IoT applications: A survey on resource-efficient distributed artificial intelligence. IEEE Commun. Surv. & Tutor. 24 (4), 2366–2418.

Balaskas, K., Karatzas, A., Sad, C., Siozios, K., Anagnostopoulos, I., Zervakis, G., et al., 2024. Hardware-aware DNN compression via diverse pruning and mixed-precision quantization. IEEE Trans. Emerg. Top. Comput..

Benbarrad, T., Salhaoui, M., Kenitar, S.B., Arioua, M., 2021. Intelligent machine vision model for defective product inspection based on machine learning. J. Sens. Actuator Netw. 10 (1), 7.

Bera, S., Dey, T., Mukherjee, A., De, D., 2024. FLAG: Federated learning for sustainable irrigation in agriculture 5.0. IEEE Trans. Consum. Electron..

Bharti, S., Mcgibney, A., 2021. Privacy-aware resource sharing in cross-device federated model training for collaborative predictive maintenance. IEEE Access 9, 120367–120379.

Bharti, S., McGibney, A., 2022. CoRoL: A reliable framework for computation offloading in collaborative robots. IEEE Internet Things J. 9 (19), 18195–18207.

Brik, B., Messaadia, M., Sahnoun, M., Bettayeb, B., Benatia, M.A., 2022. Fog-supported low-latency monitoring of system disruptions in industry 4.0: A federated learning approach. ACM Trans. Cyber-Phys. Syst. (TCPS) 6 (2), 1–23.

Build and scale the next wave of AI innovation on AWS, 2025. https://aws.amazon.com/ai/.

Cao, K., Li, L., Cui, Y., Wei, T., Hu, S., 2020. Exploring placement of heterogeneous edge servers for response time minimization in mobile edge-cloud computing. IEEE Trans. Ind. Inform. 17 (1), 494–503.

Chang, L., Deng, X., Pan, J., Zhang, Y., 2021a. Edge server placement for vehicular ad hoc networks in metropolitans. IEEE Internet Things J. 9 (2), 1575–1590.

Chang, Z., Liu, S., Xiong, X., Cai, Z., Tu, G., 2021b. A survey of recent advances in edge-computing-powered artificial intelligence of things. IEEE Internet Things J. 8 (18), 13849–13875.

Chen, Z., Hu, J., Chen, X., Hu, J., Zheng, X., Min, G., 2020. Computation offloading and task scheduling for DNN-based applications in cloud-edge computing. IEEE Access 8, 115537–115547.

Chen, X., Li, M., Zhong, H., Ma, Y., Hsu, C.-H., 2021a. DNNOff: Offloading DNN-based intelligent IoT applications in mobile edge computing. IEEE Trans. Ind. Inform. 18 (4), 2820–2829.

Chen, Y., Lin, Y., Zheng, Z., Yu, P., Shen, J., Guo, M., 2022a. Preference-aware edge server placement in the internet of things. IEEE Internet Things J. 9 (2), 1289–1299.

Chen, B., Shen, C., Wang, D., Kong, L., Chen, L., Zhu, Z., 2022b. A lifelong learning method for gearbox diagnosis with incremental fault types. IEEE Trans. Instrum. Meas. 71, 1–10.

Chen, J., Van Le, D., Tan, R., Ho, D., 2023a. NNFacet: Splitting neural network for concurrent smart sensors. IEEE Trans. Mob. Comput. 23 (2), 1627–1640.

Chen, Y., Wang, D., Wu, N., Xiang, Z., 2023b. Mobility-aware edge server placement for mobile edge computing. Comput. Commun. 208, 136–146.

Chen, Y., Yang, Q., He, S., Shi, Z., Chen, J., Guizani, M., 2023c. Ftpipehd: A fault-tolerant pipeline-parallel distributed training approach for heterogeneous edge devices. IEEE Trans. Mob. Comput. 23 (4), 3200–3212.

Chen, X., Zhang, J., Lin, B., Chen, Z., Wolter, K., Min, G., 2021b. Energy-efficient offloading for DNN-based smart IoT systems in cloud-edge environments. IEEE Trans. Parallel Distrib. Syst. 33 (3), 683–697.

Cheng, Z., Xia, X., Liwang, M., Fan, X., Sun, Y., Wang, X., Huang, L., 2023. Cheese: Distributed clustering-based hybrid federated split learning over edge networks. IEEE Trans. Parallel Distrib. Syst..

Cheng, H., Zhang, M., Shi, J.Q., 2024. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. IEEE Trans. Pattern Anal. Mach. Intell..

Chin, T.-L., Chen, Y.-S., Lyu, K.-Y., 2020. Queuing model based edge placement for work offloading in mobile cloud networks. IEEE Access 8, 47295–47303.

Craig, L., 2018. How data will build the factories of the future. https://www.techerati.com/the-stack-archive/data-centre/2018/05/14/smart-manufacturing-factory-automation/.

Cyber-Physical Systems, https://www.izm.fraunhofer.de/en/feature_topics/cyber-physical-systems.html.

Cyber-Physical Systems, 2019. https://plattformindustrie40.at/forschung-entwicklung-innovation/cyber-physische-systeme/.

De Rango, F., Guerrieri, A., Raimondo, P., Spezzano, G., 2023. HED-FL: A hierarchical, energy efficient, and dynamic approach for edge federated learning. Pervasive Mob. Comput. 92, 101804.
Deng, X., Li, J., Ma, C., Wei, K., Shi, L., Ding, M., Chen, W., 2022. Low-latency federated learning with DNN partition in distributed industrial IoT networks. IEEE J. Sel. Areas Commun. 41 (3), 755–775.
Deng, S., Zhao, H., Fang, W., Yin, J., Dustdar, S., Zomaya, A.Y., 2020. Edge intelligence: The confluence of edge computing and artificial intelligence. IEEE Internet Things J. 7 (8), 7457–7469.
Djenouri, Y., Michalak, T.P., Lin, J.C.-W., 2023. Federated deep learning for smart city edge-based applications. Future Gener. Comput. Syst. 147, 350–359.
Dong, C., Shafiq, M., Al Dabel, M.M., Sun, Y., Tian, Z., 2023. DNN inference acceleration for smart devices in industry 5.0 by decentralized deep reinforcement learning. IEEE Trans. Consum. Electron..
Dong, S., Wang, P., Abbas, K., 2021. A survey on deep learning and its applications. Comput. Sci. Rev. 40, 100379.
Dong, F., Wang, H., Shen, D., Huang, Z., He, Q., Zhang, J., Wen, L., Zhang, T., 2022. Multi-exit DNN inference acceleration based on multi-dimensional optimization for edge intelligence. IEEE Trans. Mob. Comput. 22 (9), 5389–5405.
Duan, S., Wang, D., Ren, J., Lyu, F., Zhang, Y., Wu, H., Shen, X., 2022. Distributed artificial intelligence empowered by end-edge-cloud computing: A survey. IEEE Commun. Surv. & Tutor. 25 (1), 591–624.
Duan, Y., Wu, J., 2023. Optimizing job offloading schedule for collaborative DNN inference. IEEE Trans. Mob. Comput. 23 (4), 3436–3451.
Fan, Q., Ansari, N., 2019. On cost aware cloudlet placement for mobile edge computing. IEEE/CAA J. Autom. Sin. 6 (4), 926–937.
Fan, W., Gao, L., Su, Y., Wu, F., Liu, Y., 2023. Joint DNN partition and resource allocation for task offloading in edge–cloud-assisted IoT environments. IEEE Internet Things J. 10 (12), 10146–10159.
Fang, W., Xue, F., Ding, Y., Xiong, N., Leung, V.C., 2020. EdgeKE: An on-demand deep learning IoT system for cognitive big data on industrial edge devices. IEEE Trans. Ind. Inform. 17 (9), 6144–6152.
Farahani, B., Monsefi, A.K., 2023. Smart and collaborative industrial IoT: A federated learning and data space approach. Digit. Commun. Netw. 9 (2), 436–447.
Feltin, T., Marchó, L., Cordero-Fuertes, J.-A., Brockners, F., Clausen, T.H., 2023. Dnn partitioning for inference throughput acceleration at the edge. IEEE Access 11, 52236–52249.
Feng, X., Luo, C., Chen, J., Huang, Y., Zhang, J., Xu, W., Li, J., Leung, V.C., 2023. IoTSL: Toward efficient distributed learning for resource-constrained internet of things. IEEE Internet Things J. 10 (11), 9892–9905.
Gao, M., Shen, R., Shi, L., Qi, W., Li, J., Li, Y., 2021. Task partitioning and offloading in DNN-task enabled mobile edge computing networks. IEEE Trans. Mob. Comput. 22 (4), 2435–2445.
Gauttam, H., Pattanaik, K., Bhadauria, S., Nain, G., 2022a. Bayesian network based reliability analysis in edge computing enabled machine vision system. In: 2022 IEEE Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation. IATMSI, IEEE, pp. 1–6.
Gauttam, H., Pattanaik, K., Bhadauria, S., Nain, G., 2022b. A data traffic remapping scheme for delay sensitive industrial applications. In: TENCON 2022-2022 IEEE Region 10 Conference. TENCON, IEEE, pp. 1–6.
Gauttam, H., Pattanaik, K., Bhadauria, S., Nain, G., 2024. SCL: A sustainable deep learning solution for edge computing ecosystem in smart manufacturing. J. Ind. Inf. Integr. 42, 100703.
Gauttam, H., Pattanaik, K.K., Bhadauria, S., Nain, G., Prakash, P.B., 2023. An efficient DNN splitting scheme for edge-AI enabled smart manufacturing. J. Ind. Inf. Integr. 34, 100481.
Gauttam, H., Pattanaik, K., Bhadauria, S., Saxena, D., et al., 2022c. A cost aware topology formation scheme for latency sensitive applications in edge infrastructure-as-a-service paradigm. J. Netw. Comput. Appl. 199, 103303.
Ghasemzadeh, A., Aghdasi, H.S., Saeedvand, S., 2024. Edge server placement and allocation optimization: A tradeoff for enhanced performance. Clust. Comput. 1–15.
Ghosh, S.K., Raha, A., Raghunathan, V., Raghunathan, A., 2024. PArtNNer: Platform-agnostic adaptive edge-cloud DNN partitioning for minimizing end-to-end latency. ACM Trans. Embed. Comput. Syst. 23 (1), 1–38.
Gil, Y., Park, J.-H., Baek, J., Han, S., 2021. Quantization-aware pruning criterion for industrial applications. IEEE Trans. Ind. Electron. 69 (3), 3203–3213.
Gill, S.S., Golec, M., Hu, J., Xu, M., Du, J., Wu, H., Walia, G.K., Murugesan, S.S., Ali, B., Kumar, M., et al., 2025. Edge AI: A taxonomy, systematic review and future directions. Clust. Comput. 28 (1), 1–53.
Gou, J., Yu, B., Maybank, S.J., Tao, D., 2021. Knowledge distillation: A survey. Int. J. Comput. Vis. 129 (6), 1789–1819.
Guo, Y., Wang, S., Zhou, A., Xu, J., Yuan, J., Hsu, C.-H., 2020. User allocation-aware edge cloud placement in mobile edge computing. Softw.: Pr. Exp. 50 (5), 489–502.
Guo, J., Wu, J., Liu, A., Xiong, N.N., 2021. LightFed: An efficient and secure federated edge learning system on model splitting. IEEE Trans. Parallel Distrib. Syst. 33 (11), 2701–2713.
Guo, J., Xu, C., Ling, Y., Liu, Y., Yu, Q., 2023. Latency minimization for split federated learning. In: 2023 IEEE 98th Vehicular Technology Conference (VTC2023-Fall). IEEE, pp. 1–6.
Gupta, O., Raskar, R., 2018. Distributed learning of deep neural network over multiple agents. J. Netw. Comput. Appl. 116, 1–8.
Hamad, R.K., 2025. Integrating machine learning and genetic algorithms to enhance gene-disease classification: An xbnet-based framework. Babylon. J. Mach. Learn. 2025, 1–12.
Han, S., Mao, H., Dally, W.J., 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. arXiv preprint arXiv:1510.00149.
He, F., Liu, C., Wang, M., Yang, E., Liu, X., 2023. Network lightweight method based on knowledge distillation is applied to RV reducer fault diagnosis. Meas. Sci. Technol. 34 (9), 095110.
He, Y., Zhang, X., Sun, J., 2017. Channel pruning for accelerating very deep neural networks. In: Proceedings of the IEEE International Conference on Computer Vision. pp. 1389–1397.
Holdsworth, J., Scapicchio, M., 2024. What is deep learning. https://www.ibm.com/think/topics/deep-learning.
Hou, S., Dong, B., Wang, H., Wu, G., 2020. Inspection of surface defects on stay cables using a robot and transfer learning. Autom. Constr. 119, 103382.
Hu, L., Sun, G., Ren, Y., 2020. CoEdge: Exploiting the edge-cloud collaboration for faster deep learning. IEEE Access 8, 100533–100541.
Hua, H., Li, Y., Wang, T., Dong, N., Li, W., Cao, J., 2023. Edge computing with artificial intelligence: A machine learning perspective. ACM Comput. Surv. 55 (9), 1–35.
Huang, B., Huang, X., Liu, X., Ding, C., Yin, Y., Deng, S., 2024. Adaptive partitioning and efficient scheduling for distributed dnn training in heterogeneous IoT environment. Comput. Commun. 215, 169–179.
Huang, Y., Qiao, X., Lai, W., Dustdar, S., Zhang, J., Li, J., 2021. Enabling DNN acceleration with data and model parallelization over ubiquitous end devices. IEEE Internet Things J. 9 (16), 15053–15065.
Jia, M., Cao, J., Liang, W., 2017. Optimal cloudlet placement and user to cloudlet allocation in wireless metropolitan area networks. IEEE Trans. Cloud Comput. 5 (4), 725–737.
Jia, Y., Liu, B., Dou, W., Xu, X., Zhou, X., Qi, L., Yan, Z., 2022. CroApp: A CNN-based resource optimization approach in edge computing environment. IEEE Trans. Ind. Inform. 18 (9), 6300–6307.
Jiang, X., Hou, P., Zhu, H., Li, B., Wang, Z., Ding, H., 2023. Dynamic and intelligent edge server placement based on deep reinforcement learning in mobile edge computing. Ad Hoc Netw. 145, 103172.
Jiang, C., Wan, J., Abbas, H., 2020. An edge computing node deployment method based on improved k-means clustering algorithm for smart manufacturing. IEEE Syst. J. 15 (2), 2230–2240.
Jiao, Z., Huang, K., Wang, Q., Zhong, Z., Cai, Y., 2024. Real-time litchi detection in complex orchard environments: A portable, low-energy edge computing approach for enhanced automated harvesting. Artif. Intell. Agric. 11, 13–22.
Jouini, O., Sethom, K., Namoun, A., Aljohani, N., Alanazi, M.H., Alanazi, M.N., 2024. A survey of machine learning in edge computing: Techniques, frameworks, applications, issues, and research directions. Technologies 12 (6), 81.
Jung, S., Lee, H.-W., 2023. Optimization framework for splitting DNN inference jobs over computing networks. Comput. Netw. 232, 109814.
Kang, Y., Hauswald, J., Gao, C., Rovinski, A., Mudge, T., Mars, J., Tang, L., 2017. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. ACM SIGARCH Comput. Archit. News 45 (1), 615–629.
Kasi, S.K., Kasi, M.K., Ali, K., Raza, M., Afzal, H., Lasebae, A., Naeem, B., Ul Islam, S., Rodrigues, J.J., 2020. Heuristic edge server placement in industrial internet of things and cellular networks. IEEE Internet Things J. 8 (13), 10308–10317.
Kavitha, T., Amirthayogam, G., Hephzipah, J.J., Suganthi, R., Chelladurai, T., et al., 2024. Healthcare analysis based on diabetes prediction using a cuckoo-based deep convolutional long-term memory algorithm. Babylon. J. Artif. Intell. 2024, 64–72.
Khan, W.Z., Ahmed, E., Hakak, S., Yaqoob, I., Ahmed, A., 2019. Edge computing: A survey. Future Gener. Comput. Syst. 97, 219–235.
Khan, L.U., Guizani, M., Al-Fuqaha, A., Hong, C.S., Niyato, D., Han, Z., 2023. A joint communication and learning framework for hierarchical split federated learning. IEEE Internet Things J..
Ko, H., Kim, B., Kim, Y., Pack, S., 2024. Two-phase split computing framework in edge-cloud continuum. IEEE Internet Things J..
Kook, S., Shin, W.-Y., Kim, S.-L., Ko, S.-W., 2024. Energy-efficient edge learning via joint data deepening-and-prefetching. IEEE Trans. Wirel. Commun..
Lähderanta, T., Leppänen, T., Ruha, L., Lovén, L., Harjula, E., Ylianttila, M., Riekki, J., Sillanpää, M.J., 2021. Edge computing server placement with capacitated location allocation. J. Parallel Distrib. Comput. 153, 130–149.
Le, K.-H., Le-Minh, K.-H., Thai, H.-T., 2023. Brainyedge: An ai-enabled framework for iot edge computing. ICT Express 9 (2), 211–221.
Lee, J., Lee, H., Choi, W., 2023. Wireless channel adaptive DNN split inference for resource-constrained edge devices. IEEE Commun. Lett. 27 (6), 1520–1524.
Li, X., Bi, S., 2024. Optimal AI model splitting and resource allocation for device-edge co-inference in multi-user wireless sensing systems. IEEE Trans. Wirel. Commun..
Li, C., Chai, L., Jiang, K., Zhang, Y., Liu, J., Wan, S., 2023a. DNN partition and offloading strategy with improved particle swarm genetic algorithm in VEC. IEEE Trans. Intell. Veh..
Li, W., Chen, J., Li, Y., Wen, Z., Peng, J., Wu, X., 2022a. Mobile edge server deployment towards task offloading in mobile edge computing: A clustering approach. Mob. Netw. Appl. 27 (4), 1476–1489.

Li, Y., Chen, Y., Zhu, K., Bai, C., Zhang, J., 2022b. An effective federated learning verification strategy and its applications for fault diagnosis in industrial IOT systems. IEEE Internet Things J. 9 (18), 16835–16849.
Li, H., Li, X., Fan, Q., He, Q., Wang, X., Leung, V.C., 2024a. Distributed DNN inference with fine-grained model partitioning in mobile edge computing networks. IEEE Trans. Mob. Comput..
Li, D., Li, S., Lai, Z., Fu, Y., Ye, X., Cai, L., Qiao, L., 2023b. A memory-efficient hybrid parallel framework for deep neural network training. IEEE Trans. Parallel Distrib. Syst..
Li, T., Ma, Y., Endoh, T., 2022c. From algorithm to module: Adaptive and energy-efficient quantization method for edge artificial intelligence in iot society. IEEE Trans. Ind. Inform. 19 (8), 8953–8964.
Li, L., Ota, K., Dong, M., 2018a. Deep learning for smart industry: Efficient manufacture inspection system with fog computing. IEEE Trans. Ind. Inform. 14 (10), 4665–4673.
Li, H., Ota, K., Dong, M., 2018b. Learning IoT in edge: Deep learning for the internet of things with edge computing. IEEE Netw. 32 (1), 96–101.
Li, Y., Wang, S., 2018. An energy-aware edge server placement algorithm in mobile edge computing. In: 2018 IEEE International Conference on Edge Computing. EDGE, IEEE, pp. 66–73.
Li, B., Wang, K., Xue, D., Pei, Y., 2018c. K-means based edge server deployment algorithm for edge computing environments. In: 2018 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI). IEEE, pp. 1169–1174.
Li, Y., Xie, T., Liu, C., Shi, Z., 2024b. Pseudo replay-based class continual learning for online new category anomaly detection in advanced manufacturing. IISE Trans. 1–19, just-accepted.
Li, E., Zeng, L., Zhou, Z., Chen, X., 2019. Edge AI: On-demand accelerating deep neural network inference via edge computing. IEEE Trans. Wirel. Commun. 19 (1), 447–457.
Li, Y., Zhou, A., Ma, X., Wang, S., 2021. Profit-aware edge server placement. IEEE Internet Things J. 9 (1), 55–67.
Lian, Z., Zeng, Q., Wang, W., Xu, D., Meng, W., Su, C., 2023. Traffic sign recognition using optimized federated learning in internet of vehicles. IEEE Internet Things J..
Liang, T., Glossner, J., Wang, L., Shi, S., Zhang, X., 2021a. Pruning and quantization for deep neural network acceleration: A survey. Neurocomputing 461, 370–403.
Liang, Y., Li, W., Lu, X., Wang, S., 2021b. Fog computing and convolutional neural network enabled prognosis for machining process optimization. Data Driven Smart Manuf. Technol. Appl. 13–35.
Liang, H., Sang, Q., Hu, C., Cheng, D., Zhou, X., Wang, D., Bao, W., Wang, Y., 2023. DNN surgery: Accelerating DNN inference on the edge through layer partitioning. IEEE Trans. Cloud Comput. 11 (3), 3111–3125.
Lien, S.-Y., Yeh, C.-H., Deng, D.-J., 2024. Optimum splitting computing for DNN training through next generation smart networks: A multi-tier deep reinforcement learning approach. Wirel. Netw. 30 (3), 1737–1751.
Lim, J.-A., Lee, J., Kwak, J., Kim, Y., 2024. Cutting-edge inference: Dynamic DNN model partitioning and resource scaling for mobile AI. IEEE Trans. Serv. Comput..
Lin, S.-Y., Du, Y., Ko, P.-C., Wu, T.-J., Ho, P.-T., Sivakumar, V., et al., 2020. Fog computing based hybrid deep learning framework in effective inspection system for smart manufacturing. Comput. Commun. 160, 636–642.
Ling, C., Feng, Z., Xu, L., Huang, Q., Zhou, Y., Zhang, W., Yadav, R., 2022. An edge server placement algorithm based on graph convolution network. IEEE Trans. Veh. Technol. 72 (4), 5224–5239.
Liu, Y., Chen, B., Wang, D., Kong, L., Shi, J., Shen, C., 2023a. A lifelong learning method based on generative feature replay for bearing diagnosis with incremental fault types. IEEE Trans. Instrum. Meas. 72, 1–11.
Liu, X., Deng, Y., Mahmoodi, T., 2022. Wireless distributed learning: A new hybrid split and federated learning approach. IEEE Trans. Wirel. Commun. 22 (4), 2650–2665.
Liu, K., Liu, C., Yan, G., Lee, V.C., Cao, J., 2023b. Accelerating dnn inference with reliability guarantee in vehicular edge computing. IEEE/ACM Trans. Netw. 31 (6), 3238–3253.
Lockner, Y., Hopmann, C., 2021. Induced network-based transfer learning in injection molding for process modelling and optimization with artificial neural networks. Int. J. Adv. Manuf. Technol. 112 (11), 3501–3513.
Lu, F., Zhou, G., Liu, Y., Zhang, C., 2022. Ensemble transfer learning for cutting energy consumption prediction of aviation parts towards green manufacturing. J. Clean. Prod. 331, 129920.
Luo, J.-H., Wu, J., Lin, W., 2017. Thinet: A filter level pruning method for deep neural network compression. In: Proceedings of the IEEE International Conference on Computer Vision. pp. 5058–5066.
Luo, L., Zhang, X., 2024. Federated split learning via mutual knowledge distillation. IEEE Trans. Netw. Sci. Eng..
Mahmoud, A.S., Lamouchi, O., Belghith, S., 2024. Advancements in machine learning and deep learning for early diagnosis of chronic kidney diseases: A comprehensive review. Babylon. J. Mach. Learn. 2024, 149–156.
Mamba Kabala, D., Hafiane, A., Bobelin, L., Canals, R., 2023. Image-based crop disease detection with federated learning. Sci. Rep. 13 (1), 19220.
Maschler, B., Tatiyosyan, S., Weyrich, M., 2022. Regularization-based continual learning for fault prediction in lithium-ion batteries. Procedia CIRP 112, 513–518.
Maschler, B., Vietz, H., Jazdi, N., Weyrich, M., 2020. Continual learning of fault prediction for turbofan engines using deep learning with elastic weight consolidation. In: 2020 25th IEEE International Conference on Emerging Technologies and Factory Automation. ETFA, vol. 1, IEEE, pp. 959–966.
Mathisen, D., D., K., 2020. Infrastructure for AI: Why storage matters. https://www.ibm.com/think/insights/infrastructure-for-ai-why-storage-matters.
Matsubara, Y., Callegaro, D., Baidya, S., Levorato, M., Singh, S., 2020. Head network distillation: Splitting distilled deep neural networks for resource-constrained edge computing systems. IEEE Access 8, 212177–212193.
Matthew, W., 2020. IoT edge computing – what it is and how it is becoming more intelligent. https://iot-analytics.com/iot-edge-computing-what-it-is-and-how-it-is-becoming-more-intelligent/.
Mazloomi, A., Sami, H., Bentahar, J., Otrok, H., Mourad, A., 2022. Reinforcement learning framework for server placement and workload allocation in multiaccess edge computing. IEEE Internet Things J. 10 (2), 1376–1390.
Mehta, A., Verma, R.K., 2024. Qos-aware edge server placement for collaborative predictive maintenance in industrial internet of things. J. Supercomput. 1–27.
Meng, J., Shi, W., Tan, H., Li, X., 2017. Cloudlet placement and minimum-delay routing in cloudlet computing. In: 2017 3rd International Conference on Big Data Computing and Communications. BIGCOM, IEEE, pp. 297–304.
Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., et al., 2017. Mixed precision training. arXiv preprint arXiv:1710.03740.
Mitch, T., Todd, E., Lalit, C., 2018. Introduction to edge computing in iIoT. https://www.iiconsortium.org/white-papers/edge-computing/.
Murshed, M.S., Murphy, C., Hou, D., Khan, N., Ananthanarayanan, G., Hussain, F., 2021. Machine learning at the network edge: A survey. ACM Comput. Surv. 54 (8), 1–37.
Mwawado, R., Zennaro, M., Nsenga, J., Hanyurwimfura, D., 2024. Optimizing soil-based crop recommendations with federated learning on raspberry pi edge computing nodes. In: 2024 11th International Conference on Internet of Things: Systems, Management and Security. IOTSMS, IEEE, pp. 82–89.
Nain, G., Pattanaik, K., Sharma, G., 2022. Towards edge computing in intelligent manufacturing: Past, present and future. J. Manuf. Syst. 62, 588–611.
Nain, G., Pattanaik, K.K., Sharma, G.K., 2024a. A novel semi-supervised learning for industrial edge computing platforms in quality prediction. SN Comput. Sci. 5 (5), 1–16.
Nain, G., Pattanaik, K.K., Sharma, G.K., Gauttam, H., 2024b. PackMASNet: An information integration approach for quality inspection in industry 5.0. Expert Syst. Appl. 255, 124582.
Nain, G., Pattanaik, K., Sharma, G., Gauttam, H., 2026. MCMDA: A continual learning and frugal AI based quality inspection mechanism for edge computing platforms. Adv. Eng. Inform. 69, 103929.
Nain, G., Pattanaik, K., Sharma, G., Gauttam, H., Viriyasitavat, W., 2023. A novel mechanism for continual learning based predictive quality inspection in smart manufacturing. In: TENCON 2023-2023 IEEE Region 10 Conference. TENCON, IEEE, pp. 606–611.
Nain, G., Samal, U., 2025. Concurrence of artificial intelligence and additive manufacturing: A bibliometric analysis. Total. Qual. Manag. Bus. Excell. 1–33.
Naveen, S., Kounte, M.R., Ahmed, M.R., 2021. Low latency deep learning inference model for distributed intelligent IoT edge clusters. IEEE Access 9, 160607–160621.
Ni, W., Ao, H., Tian, H., Eldar, Y.C., Niyato, D., 2024. FedSL: Federated split learning for collaborative healthcare analytics on resource-constrained wearable IoMT devices. IEEE Internet Things J..
Oh, S., Kim, D., Lee, C., Jeong, J., 2022. Edge-cloud alarm level of heterogeneous iIoT devices based on knowledge distillation in smart manufacturing. Electronics 11 (6), 899.
Paluri, K.V., Nain, G., Gupta, A., 2025. Advancements in acne detection and classification: A bibliometric analysis. Comput. Electr. Eng. 128, 110693.
Pan, S.J., Yang, Q., 2009. A survey on transfer learning. IEEE Trans. Knowl. Data Eng. 22 (10), 1345–1359.
Pervej, M.F., Jin, R., Dai, H., 2023. Resource constrained vehicular edge federated learning with highly mobile connected vehicles. IEEE J. Sel. Areas Commun. 41 (6), 1825–1844.
Putra, M.A.P., Rachmawati, S.M., Abisado, M., Sampedro, G.A., 2023. HFTL: Hierarchical federated transfer learning for secure and efficient fault classification in additive manufacturing. IEEE Access 11, 54795–54807.
Putrada, A.G., Abdurohman, M., Perdana, D., Nuha, H.H., 2023. Edgesl: Edge-computing architecture on smart lighting control with distilled knn for optimum processing time. IEEE Access 11, 64697–64712.
Qi, H., Ren, F., Wang, L., Jiang, P., Wan, S., Deng, X., 2024. Multi-compression scale DNN inference acceleration based on cloud-edge-end collaboration. ACM Trans. Embed. Comput. Syst. 23 (1), 1–25.
Qiu, T., Chi, J., Zhou, X., Ning, Z., Atiquzzaman, M., Wu, D.O., 2020. Edge computing in industrial internet of things: Architecture, advances and challenges. IEEE Commun. Surv. & Tutor. 22 (4), 2462–2488. http://dx.doi.org/10.1109/COMST.2020.3009103.
Rakib, A.F., Rahman, R., Razi, A.A., Hasan, A.T., 2024. A lightweight quantized CNN model for plant disease recognition. Arab. J. Sci. Eng. 49 (3), 4097–4108.

Resende, C., Folgado, D., Oliveira, J., Franco, B., Moreira, W., Oliveira-Jr, A., Cavaleiro, A., Carvalho, R., 2021. TIP4. 0: Industrial internet of things platform for predictive maintenance. Sensors 21 (14), 4676.

Richter, F., 2017. Big data on wheels. https://www.statista.com/chart/8018/connected-car-data-generation/?utm_source=chatgpt.com.

Rosendo, D., Costan, A., Valduriez, P., Antoniu, G., 2022. Distributed intelligence on the edge-to-cloud continuum: A systematic literature review. J. Parallel Distrib. Comput. 166, 71–94.

Saguil, D., Azim, A., 2020. A layer-partitioning approach for faster execution of neural network-based embedded applications in edge networks. IEEE Access 8, 59456–59469.

Salih, B.M., Mohammad, O.K.J., 2024. Cloud data leakage, security, privacy issues and challenges. Procedia Comput. Sci. 242, 592–601.

Samikwa, E., Di Maio, A., Braun, T., 2022. Ares: Adaptive resource-aware split learning for internet of things. Comput. Netw. 218, 109380.

Samikwa, E., Di Maio, A., Braun, T., 2023. Disnet: Distributed micro-split deep learning in heterogeneous dynamic iot. IEEE Internet Things J..

Shan, N., Ye, Z., Cui, X., 2020. Collaborative intelligence: Accelerating deep neural network inference via device-edge synergy. Secur. Commun. Netw. 2020 (1), 8831341.

Sharma, R., Kashyap, N., 2024. Edge AI market. https://www.rootsanalysis.com/edge-ai-market.

Shi, L., Xu, Z., Sun, Y., Shi, Y., Fan, Y., Ding, X., 2021. A DNN inference acceleration algorithm combining model partition and task allocation in heterogeneous edge computing system. Peer-To-Peer Netw. Appl. 14 (6), 4031–4045.

Shi, Y., Yang, K., Jiang, T., Zhang, J., Letaief, K.B., 2020. Communication-efficient edge AI: Algorithms and systems. IEEE Commun. Surv. & Tutor. 22 (4), 2167–2191.

Shuvo, M.M.H., Islam, S.K., Cheng, J., Morshed, B.I., 2022. Efficient acceleration of deep learning inference on resource-constrained edge devices: A review. Proc. IEEE 111 (1), 42–91.

Sinha, S., 2024. The top 6 edge AI trends—as showcased at embedded world 2024. https://www.rootsanalysis.com/edge-ai-market.

SK, P., Kesanapalli, S.A., Simmhan, Y., 2022. Characterizing the performance of accelerated jetson edge devices for training deep learning models. Proc. ACM Meas. Anal. Comput. Syst. 6 (3), 1–26.

Sourabh, B., 2021. Bring AI to your data and improve vision-based product quality inspection. https://connectamericas.com/content/bring-ai-your-data-and-improve-vision-based-product-quality-inspection.

Su, Y., Fan, W., Gao, L., Qiao, L., Liu, Y., Wu, F., 2022. Joint DNN partition and resource allocation optimization for energy-constrained hierarchical edge-cloud systems. IEEE Trans. Veh. Technol. 72 (3), 3930–3944.

Sun, W., Liu, J., Yue, Y., 2019. AI-enhanced offloading in edge computing: When machine learning meets industrial IoT. IEEE Netw. 33 (5), 68–74.

Sung, J., Han, S.-j., 2024. Use of edge resources for DNN model maintenance in 5G IoT networks. Clust. Comput. 1–13.

Tang, S., Chen, L., He, K., Xia, J., Fan, L., Nallanathan, A., 2022. Computational intelligence and deep learning for next-generation edge-enabled industrial IoT. IEEE Trans. Netw. Sci. Eng. 10 (5), 2881–2893.

Tanghatari, E., Kamal, M., Afzali-Kusha, A., Pedram, M., 2022. Distributing DNN training over IoT edge devices based on transfer learning. Neurocomputing 467, 56–65.

Taylor, P., 2024. Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2023, with forecasts from 2024 to 2028. https://www.statista.com/statistics/871513/worldwide-data-created/.

Teerapittayanon, S., McDanel, B., Kung, H.-T., 2016. Branchynet: Fast inference via early exiting from deep neural networks. In: 2016 23rd International Conference on Pattern Recognition. ICPR, IEEE, pp. 2464–2469.

Tercan, H., Deibert, P., Meisen, T., 2022. Continual learning of neural networks for quality prediction in production using memory aware synapses and weight transfer. J. Intell. Manuf. 33 (1), 283–292.

Tham, C.-K., Sharma, N., 2021. Prescriptive maintenance using Markov decision process and GPU-accelerated edge computing. Mob. Edge Comput. 167–181.

Tiwari, V., Pandey, C., Dahal, A., Roy, D.S., Fiore, U., 2024. A knapsack-based metaheuristic for edge server placement in 5G networks with heterogeneous edge capacities. Future Gener. Comput. Syst. 153, 222–233.

Tonellotto, N., Gotta, A., Nardini, F.M., Gadler, D., Silvestri, F., 2021. Neural network quantization in federated learning at the edge. Inform. Sci. 575, 417–436.

Tuli, S., Mirhakimi, F., Pallewatta, S., Zawad, S., Casale, G., Javadi, B., Yan, F., Buyya, R., Jennings, N.R., 2023. AI augmented edge and fog computing: Trends and challenges. J. Netw. Comput. Appl. 216, 103648.

Vaigandla, K.K., 2025. Role of IoT and ML in healthcare. Babylon. J. Artif. Intell. 2025, 23–36.

Vailshery, L.S., 2024. Number of internet of things (IoT) connections worldwide from 2022 to 2023, with forecasts from 2024 to 2033. https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/.

Vali, A.A., Azizi, S., Shojafar, M., 2024. RESP: A recursive clustering approach for edge server placement in mobile edge computing. ACM Trans. Internet Technol..

Vipra, J., West, S.M., 2023. Computational power and AI. https://ainowinstitute.org/publication/policy/compute-and-ai.

Wang, F., Cai, S., Lau, V.K., 2023a. Sequential offloading for distributed DNN computation in multiuser MEC systems. IEEE Internet Things J. 10 (20), 18315–18329.

Wang, E., Davis, J.J., Moro, D., Zielinski, P., Lim, J.J., Coelho, C., Chatterjee, S., Cheung, P.Y., Constantinides, G.A., 2023b. Enabling binary neural network training on the edge. ACM Trans. Embed. Comput. Syst. 22 (6), 1–19.

Wang, Z., Gao, F., Jin, X., 2020a. Optimal deployment of cloudlets based on cost and latency in internet of things networks. Wirel. Netw. 26 (8), 6077–6093.

Wang, X., Han, Y., Leung, V.C., Niyato, D., Yan, X., Chen, X., 2020b. Convergence of edge computing and deep learning: A comprehensive survey. IEEE Commun. Surv. & Tutor. 22 (2), 869–904.

Wang, J., Li, D., Hu, Y., 2020c. Fog nodes deployment based on space–time characteristics in smart factory. IEEE Trans. Ind. Inform. 17 (5), 3534–3543.

Wang, Z., Liu, Q., Chen, H., Chu, X., 2021. A deformable CNN-DLSTM based transfer learning method for fault diagnosis of rolling bearing under multiple working conditions. Int. J. Prod. Res. 59 (16), 4811–4825.

Wang, Q., Liu, B., Li, Y., 2024a. Traceable federated continual learning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 12872–12881.

Wang, J., Wu, Y., Chen, Y.-Q., 2023c. Branchy deep learning based real-time defect detection under edge-cloud fusion architecture. IEEE Trans. Cloud Comput. 11 (3), 3301–3313.

Wang, L., Xu, Y., Xu, H., Chen, M., Huang, L., 2022a. Accelerating decentralized federated learning in heterogeneous edge computing. IEEE Trans. Mob. Comput. 22 (9), 5001–5016.

Wang, Z., Yang, G., Dai, H., Rong, C., 2023d. Privacy-preserving split learning for large-scaled vision pre-training. IEEE Trans. Inf. Forensics Secur. 18, 1539–1553.

Wang, Y., Yang, C., Lan, S., Zhu, L., Zhang, Y., 2024b. End-edge-cloud collaborative computing for deep learning: A comprehensive survey. IEEE Commun. Surv. & Tutor..

Wang, C., Yang, G., Papanastasiou, G., Zhang, H., Rodrigues, J.J., De Albuquerque, V.H.C., 2020d. Industrial cyber-physical systems-based cloud IoT edge for federated heterogeneous distillation. IEEE Trans. Ind. Inform. 17 (8), 5511–5521.

Wang, Z., Zhang, W., Jin, X., Huang, Y., Lu, C., 2022b. An optimal edge server placement approach for cost reduction and load balancing in intelligent manufacturing. J. Supercomput. 78 (3), 4032–4056.

Wang, L., Zhang, X., Su, H., Zhu, J., 2024c. A comprehensive survey of continual learning: Theory, method and application. IEEE Trans. Pattern Anal. Mach. Intell..

Wang, S., Zhao, Y., Xu, J., Yuan, J., Hsu, C.-H., 2019. Edge server placement in mobile edge computing. J. Parallel Distrib. Comput. 127, 160–168.

What Is Industrial IoT (IIoT)?, 2025. https://www.cisco.com/c/en/us/solutions/internet-of-things/what-is-industrial-iot.html.

Wu, L., Gao, G., Yu, J., Zhou, F., Yang, Y., Wang, T., 2023. PDD: Partitioning DAG-topology DNNs for streaming tasks. IEEE Internet Things J..

Wu, D., Ullah, R., Rodgers, P., Kilpatrick, P., Spence, I., Varghese, B., 2024a. EcoFed: Efficient communication for DNN partitioning-based federated learning. IEEE Trans. Parallel Distrib. Syst..

Wu, M., Yang, R., Huang, X., Wu, Y., Kang, J., Xie, S., 2024b. Joint optimization of model partition and resource allocation for split federated learning over vehicular edge networks. IEEE Trans. Veh. Technol..

Xia, M., Shao, H., Williams, D., Lu, S., Shu, L., de Silva, C.W., 2021. Intelligent fault diagnosis of machinery using digital twin-assisted deep transfer learning. Reliab. Eng. Syst. Saf. 215, 107938.

Xu, D., Li, T., Li, Y., Su, X., Tarkoma, S., Jiang, T., Crowcroft, J., Hui, P., 2021. Edge intelligence: Empowering intelligence to the edge of network. Proc. IEEE 109 (11), 1778–1837.

Xu, C., Li, J., Liu, Y., Ling, Y., Wen, M., 2023a. Accelerating split federated learning over wireless communication networks. IEEE Trans. Wirel. Commun..

Xu, Y., Mohammed, T., Di Francesco, M., Fischione, C., 2022. Distributed assignment with load balancing for dnn inference at the edge. IEEE Internet Things J. 10 (2), 1053–1065.

Xu, M., Qian, F., Zhu, M., Huang, F., Pushp, S., Liu, X., 2019. Deepwear: Adaptive local offloading for on-wearable deep learning. IEEE Trans. Mob. Comput. 19 (2), 314–330.

Xu, X., Yan, K., Han, S., Wang, B., Tao, X., Zhang, P., 2023b. Learning-based edge-device collaborative dnn inference in iovt networks. IEEE Internet Things J..

Xue, M., Wu, H., Li, R., Xu, M., Jiao, P., 2021. EosDNN: An efficient offloading scheme for DNN inference acceleration in local-edge-cloud collaborative environments. IEEE Trans. Green Commun. Netw. 6 (1), 248–264.

Yang, B., Fagbohungbe, O., Cao, X., Yuen, C., Qian, L., Niyato, D., Zhang, Y., 2021a. A joint energy and latency framework for transfer learning over 5G industrial edge networks. IEEE Trans. Ind. Inform. 18 (1), 531–541.

Yang, S., He, S., Duan, H., Chen, W., Zhang, X., Wu, T., Yin, Y., 2023. APQ: Automated DNN pruning and quantization for ReRAM-based accelerators. IEEE Trans. Parallel Distrib. Syst. 34 (9), 2498–2511.

Yang, S., Li, F., Shen, M., Chen, X., Fu, X., Wang, Y., 2019. Cloudlet placement and task allocation in mobile edge computing. IEEE Internet Things J. 6 (3), 5853–5863.

Yang, Y.-T., Wei, H.-Y., 2022. A coalition formation approach for privacy and energy-aware split deep learning inference in edge camera network. IEEE Trans. Netw. Serv. Manag. 20 (3), 3673–3685.

Yang, X., Zhang, Y., Lv, W., Wang, D., 2021b. Image recognition of wind turbine blade damage based on a deep learning model with transfer learning and an ensemble learning classifier. Renew. Energy 163, 386–397.

Yao, J., Zhang, S., Yao, Y., Wang, F., Ma, J., Zhang, J., Chu, Y., Ji, L., Jia, K., Shen, T., et al., 2022. Edge-cloud polarization and collaboration: A comprehensive survey for ai. IEEE Trans. Knowl. Data Eng. 35 (7), 6866–6886.

Yu, F., Cui, L., Wang, P., Han, C., Huang, R., Huang, X., 2020. EasiEdge: A novel global deep neural networks pruning method for efficient edge computing. IEEE Internet Things J. 8 (3), 1259–1271.

Yu, Y.-C., Yang, S.-R., Chuang, S.-W., Chien, J.-T., Lee, C.-Y., 2024. Pruning quantized unsupervised meta-learning DegradingNet solution for industrial equipment and semiconductor process anomaly detection and prediction. Appl. Sci. 14 (5), 1708.

Yun, S., Choi, W., Kim, I.-M., 2022. Cooperative inference of DNNs for delay- and memory-constrained wireless IoT systems. IEEE Internet Things J. 9 (17), 16113–16127.

Zeng, L., Li, E., Zhou, Z., Chen, X., 2019. Boomerang: On-demand cooperative deep neural network inference for edge intelligence on the industrial internet of things. IEEE Netw. 33 (5), 96–103.

Zeng, F., Ren, Y., Deng, X., Li, W., 2018. Cost-effective edge server placement in wireless metropolitan area networks. Sensors 19 (1), 32.

Zeng, T., Semiari, O., Mozaffari, M., Chen, M., Saad, W., Bennis, M., 2020. Federated learning in the sky: Joint power allocation and scheduling with UAV swarms. In: ICC 2020-2020 IEEE International Conference on Communications. ICC, IEEE, pp. 1–6.

Zhang, J., Chen, C., Zhuang, W., Lyu, L., 2023a. Target: Federated class-continual learning via exemplar-free distillation. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 4782–4793.

Zhang, T., Ding, B., Zhao, X., Liu, G., Pang, Z., 2021a. LearningADD: Machine learning based acoustic defect detection in factory automation. J. Manuf. Syst. 60, 48–58.

Zhang, X., Hu, M., Xia, J., Wei, T., Chen, M., Hu, S., 2020. Efficient federated learning for cloud-based aIoT applications. IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 40 (11), 2211–2223.

Zhang, Y., Huang, J., Du, L., 2017. The top-cited systematic reviews/meta-analyses in tuberculosis research: A PRISMA-compliant systematic literature review and bibliometric analysis. Medicine 96 (6), e4822.

Zhang, T., Li, Y., Chen, C.P., 2021b. Edge computing and its role in industrial internet: Methodologies, applications, and future directions. Inform. Sci. 557, 34–65.

Zhang, L., Xu, J., 2021. Learning the optimal partition for collaborative DNN training with privacy requirements. IEEE Internet Things J. 9 (13), 11168–11178.

Zhang, Z., Zhang, Y., Guo, D., Zhao, S., Zhu, X., 2023b. Communication-efficient federated continual learning for distributed learning system with non-IID data. Sci. China Inf. Sci. 66 (2), 122102.

Zhang, C., Zhang, W., Wu, Q., Fan, P., Fan, Q., Wang, J., Letaief, K.B., 2024a. Distributed deep reinforcement learning based gradient quantization for federated learning enabled vehicle edge computing. IEEE Internet Things J..

Zhang, W., Zhou, T., Lu, Q., Yuan, Y., Tolba, A., Said, W., 2024b. FedSL: A communication efficient federated learning with split layer aggregation. IEEE Internet Things J..

Zheng, H., Gao, M., Chen, Z., Feng, X., 2021. A distributed hierarchical deep computation model for federated learning in edge computing. IEEE Trans. Ind. Inform. 17 (12), 7946–7956.

Zhou, J., Wang, Y., Ota, K., Dong, M., 2019. AAIoT: Accelerating artificial intelligence in IoT systems. IEEE Wirel. Commun. Lett. 8 (3), 825–828.

Zhu, Z., Han, G., Jia, G., Shu, L., 2020. Modified densenet for automatic fabric defect detection with edge computing for minimizing latency. IEEE Internet Things J. 7 (10), 9623–9636.

Zim, A.H., Iqbal, A., Hossain, L., Arif, S., Malik, A., Rasool, I., Kuribayashi, M., Ahmad, F., 2023. Smart manufacturing with transfer learning under limited data: Towards data-driven intelligences. Mater. Today Commun. 37, 107357.

Zizzo, G., Rawat, A., Holohan, N., Tirupathi, S., 2022. Federated continual learning with differentially private data sharing. In: Workshop on Federated Learning: Recent Advances and New Challenges (in Conjunction with NeurIPS 2022).